Integración de LLMs y grafos de conocimiento



Realizado por: Francisco Javier Borrego Caraballo

Índice

1. Introducción 5
1.1. Descripción del problema 5
1.2. Propuesta de solución 6
2. Contexto y estado del arte 7
2.1. Representación del conocimiento 7
2.2. Modelos de Lenguaje Grandes (LLMs) 9
2.2.1. Proceso de entrenamiento 9
2.2.2. Vertex AI como ejemplo de LLM 9
2.2.3. Ventajas de los LLMs 10
2.2.4. Casos de uso 10
2.3. Grafos de conocimiento 10
2.3.1. Definición y fundamentos 10
2.3.2. Ventajas de utilizar grafos de conocimiento 11
2.4. Grafos de conocimiento en IA conversacional 12
2.4.1. Integración de grafos de conocimiento y LLMs 12
2.4.2. Ventajas de integrar LLMs y grafos de conocimiento 12
2.4.3. Casos de uso 12
2.4.4. Buenas prácticas 13
2.4.5. Recuperación de la información basada en grafos: GraphRAG 14
2.5. Explicabilidad del sistema con grafos de conocimiento 15
2.5.1. Ingeniería de prompt 15
2.6. Seguridad en bases de datos orientadas a grafo 17
2.6.1. Inyección de Cypher 17
2.6.2. Protección frente ataques 17
2.7. Revisión y análisis de artículos y revistas 18
2.7.1. Grafos de tareas para mejorar la planificación en agentes basados en LLMs . 18
2.7.2. Mejora en la recuperación precisa sobre grafos de conocimiento a gran escala 19
2.7.3. Construcción de grafos de conocimiento y evaluación del sistema 20
2.7.4. Preocupaciones de la seguridad y privacidad en el uso de LLMs 21
3. Ecosistema tecnológico 23
3.1. Herramientas software 23
3.1.1. Python 23
3.1.2. LangChain 23
3.1.3. Mistral AI 24

3.1.4. Base de datos Neo4j 25
3.1.5. Servidor Flask 26
3.1.6. Vue.js 27
3.1.7. TypeScript 27
3.2. Arquitectura general del sistema 28
3.3. Diseño e integración 29
3.3.1. Flujo de creación del grafo de conocimiento 29
3.3.2. Flujo de generación de respuestas para el usuario 30
4. Implementación técnica 31
4.1. Construcción del grafo de conocimiento 31
4.1.1. Estructura y organización de los datos 31
4.1.2. Lectura de los datos 32
4.1.3. Definición del prompt y LLM generador 32
4.1.4. Estructuración del grafo de conocimiento 35
4.1.5. Definición del prompt y LLM evaluador 35
4.1.6. Conexión y subida a la base de datos Neo4j Aura 36
4.1.7. Visualización del grafo de conocimiento transaccional 38
4.2. Integración del grafo de conocimiento con el LLM 42
4.2.1. Carga de datos y conexión con la base de datos Neo4j Aura 42
4.2.2. Configuración e instanciación para LLMs 43
4.2.3. Prompt para el LLM generador de consultas Cypher 43
4.2.4. Funciones auxiliares 45
4.2.5. Prompt para el LLM generador de respuestas en lenguaje natural 47
4.2.6. Funcionamiento del sistema conversacional LLM-usuario 47
4.3. Interfaz conversacional 48
4.3.1. Estructura general 49
4.3.2. Comunicación con el backend 50
4.3.3. Estilos y experiencia de usuario 50
4.4. Ejemplos representativos del sistema 50
4.4.1. Características del sistema 50
4.4.2. Ejemplo 1 - Interacción del usuario con el sistema 51
4.4.3. Ejemplo 2 - Consulta dependiente sin memoria 52
4.4.4. Ejemplo 3 - Consulta sobre ingresos importantes 52
4.4.5. Ejemplo 4 - Gestión de mensajes distintos a consulta de transacciones 53
4.4.6. Ejemplo 5 - Consulta en otro idioma 53
4.4.7. Ejemplo 6 – Protección frente a inyección de código Cypher 54

4.4.8. Ejemplo 7 - Pregunta sobre terminología de finanzas 54
4.4.9. Ejemplo 8 – Protección frente a persuasiones 55
4.5. Problemas y dificultades encontrados 55
4.6. Manual de instalación y configuración 58
4.6.1. Generación del grafo de conocimiento transaccional 58
4.6.2. Configuración de la aplicación web de interfaz conversacional 59
5. Evaluación del sistema 60
5.1. Exactitud de las respuestas 60
5.1.1. Conjunto de preguntas y respuestas 60
5.1.2. Comparación entre las respuestas esperadas y obtenidas 61
5.2. Robustez del sistema 62
5.2.1. Prueba sobre la pregunta con solución correcta/inválida 62
5.3. Conclusiones sobre evaluación 63
6. Conclusiones y trabajo futuro 64
7. Bibliografía 65

1. Introducción

El presente trabajo final del Máster de Lógica, Computación e Inteligencia Artificial tiene como objetivo llevar a cabo una investigación que no solo explore la naturaleza y funcionamiento de los modelos de lenguaje grandes (LLMs), sino que también profundice en su relación con los grafos de conocimiento. Se analiza cómo se definen estos grafos, su evolución hasta convertirse en una herramienta ampliamente utilizada en la actualidad, y los principales ámbitos en los que se aplican, como la medicina, las finanzas o la industria.

Para abordar este objetivo, se introducen los conceptos fundamentales, se plantea el problema principal que motiva el trabajo y se proponen posibles soluciones, sin dejar de lado aspectos clave como la arquitectura tecnológica empleada o la evaluación mediante métricas de rendimiento, tanto clásicas como innovadoras, dentro del ámbito del procesamiento del lenguaje natural.

En este contexto, se desarrolla un sistema experimental basado en un modelo de lenguaje integrado con un grafo de conocimiento, con el fin de explorar su potencial en entornos reales como el bancario, empresarial o incluso en sectores como el alimentario. Además, se analiza la capacidad de respuesta que puede dar un modelo de lenguaje grande a través de un grafo de conocimiento, analizando su potencial, así como el posible riesgo de exposición o modificación de datos sensibles en distintos escenarios.

1.1. Descripción del problema

Cuando hablamos de grafos de conocimiento, sabemos que están compuestos por nodos, que representan entidades y relaciones conocidas como aristas, que vinculan a los nodos entre sí por alguna característica. Aquí es donde entra la pregunta de cómo aprovechar dicha información de forma óptima con modelos de lenguaje grandes (LLMs), y sacar a relucir todas sus características, teniendo en cuenta que carecen de memoria estructurada y actualizada para tareas específicas.

Con ello, la elaboración del presente trabajo fin de máster consiste en abordar la relación que existe entre los LLMs y los grafos de conocimiento, en aspectos como la extracción de datos y su procesamiento, así como la seguridad que conlleva hacer uso de estos para responder a las preguntas de un usuario. En concreto, se explora el análisis de transacciones bancarias para un usuario.

El problema por abordar consiste en que el usuario quiere hacer preguntas complejas sobre sus finanzas, pero los LLMs no entienden del todo relaciones temporales, categorizaciones de gastos, detección de anomalías o proyecciones, si no están integrados con un sistema bien estructurado.

Un posible caso de uso se presenta a continuación:

El usuario pregunta en una interfaz conversacional:

- *"¿Cuánto he gastado este mes en suscripciones?"*
- *"¿Qué transacción fue más inusual el último trimestre?"*
- *"¿Mis ingresos han aumentado respecto al trimestre anterior?"*

Para resolver estas cuestiones, el LLM debe acceder a datos financieros estructurados, donde entra en juego el uso de los grafos de conocimiento.

Esto conlleva a su vez un gran riesgo, ya que, en dicho grafo, hay información muy sensible del usuario, como transacciones, pagos realizados, lugares en los que estuvo o incluso su cantidad actual de dinero. Es por ello por lo que, la investigación acerca de cómo la información es recuperada y tratada por el LLM, así como la usada para servicios externos (para publicidad personalizada entre otros), es crucial.

Otro problema adicional es el cómo el LLM 'piensa' y genera las respuestas (explicabilidad), ya que en muchas ocasiones pueden ocurrir las 'alucinaciones'. También puede darse el caso de que el usuario consulte información acerca de la transacción 19, y se le responda con la información de la transacción 2, lo cual puede parecer inofensivo a priori, pero depende el contexto o la operación que planee hacer el usuario a futuro, puede ser muy peligroso o poco conveniente. Enlazando a este problema el comportamiento del LLM (forma de expresión, filtrado de contenido no apropiado, cómo responde e incita al usuario a seguir preguntando o respondiendo, entre otras opciones).

1.2. Propuesta de solución

Debido a la inquietud de los usuarios por querer acceder a sus datos bancarios de forma segura y rápida, sin tener que buscar en incontables líneas una transacción concreta, se propone integrar un modelo conversacional (LLM) con un grafo de conocimiento (KG) que represente los datos financieros del usuario, permitiéndole interactuar de forma natural con sus finanzas, resolver consultas o detectar anomalías si el usuario hace uso indebido de la aplicación, a través de la aplicación web de interfaz conversacional.

En nuestro caso, vamos a generar el grafo de conocimiento y almacenarlo en la base de datos por separado, sin necesidad de un registro de usuario como tal, ya que la importancia del trabajo radica en el cómo y por qué el LLM es capaz de extraer la información del grafo de conocimiento, y su capacidad para responder en lenguaje natural al usuario, siendo el objetivo ofrecer una experiencia similar a la de tener un asesor financiero personal, pero accesible las 24h del día y con un costo mucho menor que el de un asesor real, gracias a la inteligencia artificial.

Aunque en el presente trabajo se emplea un conjunto de datos simulado (un archivo CSV de transacciones bancarias de un usuario ficticio), la propuesta está diseñada para escalar a entornos reales en los que el usuario, tras autenticarse en la plataforma, autoriza el acceso a sus datos bancarios mediante una API bancaria segura en donde, a partir de dicha información, se construye un grafo de conocimiento personalizado que recoge transacciones, categorías de gasto, tendencias temporales y relaciones entre entidades.

También, es conocido que, en empresas privadas, donde la seguridad de los datos es primordial, se utilizan modelos entrenados y ejecutados en local, con el fin de aumentar su privacidad. En el presente trabajo, no se trabajará bajo esa idea, puesto que los recursos son limitados (CPU y GPU), y en su lugar se opta por una API gratuita con fines de investigación.

2. Contexto y estado del arte

El uso de modelos de lenguaje grandes (LLMs) en tareas específicas ha experimentado un rápido crecimiento en los últimos años. Sin embargo, su integración con sistemas de información estructurados, como los grafos de conocimiento, sigue siendo un reto importante. En este contexto, diversos estudios han abordado la interacción entre los LLMs y datos externos, pero muchas de estas soluciones todavía presentan limitaciones en cuanto a precisión, escalabilidad y adaptabilidad a dominios especializados, como la gestión de finanzas personales.

Uno de los principales desafíos radica en la limitada capacidad de los LLMs para acceder y procesar información actualizada y estructurada, lo que limita su capacidad para realizar tareas como el análisis de grandes volúmenes de datos transaccionales. Su integración con grafos de conocimiento ha mostrado cierto potencial para resolver estos problemas, ya que permiten representar la información de manera más organizada y lógica, facilitando consultas complejas y razonamiento contextual.

También tenemos el problema de la brecha de seguridad existente en estos sistemas IA, además de muchas que aún se desconocen, sobre todo de cara a la privacidad de los usuarios, y cómo tratan las empresas sus datos, sobre todo para generar dinero con ello.

Por lo tanto, es crucial revisar la información y diversas técnicas que ya se han abordado en revistas y artículos, que sirvan de utilidad para el desarrollo y extensión del presente trabajo, así como ahondar más en el tema de la IA generativa con definición de conceptos claves, entre otros aspectos, antes de abarcar en mayor profundidad su diseño, integración y posterior implementación.

Para ello, se define de forma conceptual cada una de las definiciones, siendo las principales los grafos de conocimiento, y los lenguajes de modelo grandes (LLMs).

2.1. Representación del conocimiento

Cuando hablamos de representar el conocimiento, nos referimos a un concepto central, aunque normalmente no definido directamente, en el campo de la Inteligencia Artificial. Para comprender qué es, resulta útil analizar los cinco roles importantes y distintos que desempeña. Estos roles imponen demandas diferentes, y a veces contradictorias, sobre las propiedades que debe tener una representación, por lo que definirlos puede sernos de utilidad para el presente trabajo.

Por lo tanto, dichos roles son:

- **Un sustituto:** Representar el conocimiento puede entenderse como reemplazar esa misma cosa, para que una entidad pueda determinar consecuencias pensando en vez de actuar sobre ellas. Las operaciones realizadas sobre las representaciones sustituyen a la interacción directa con lo real. Aquí es donde entra la fidelidad perfecta, que es, en general, imposible para los objetos del mundo natural debido a su complejidad ilimitada y la necesidad de simplificar las cosas, introduciendo suposiciones o artefactos. Esta imperfección inherente implica que, al razonar ampliamente sobre el mundo natural, inevitablemente se llegará a conclusiones incorrectas, independientemente de la representación o el proceso de razonamiento empleados. Por lo tanto, una buena representación busca minimizar

el error para una tarea específica. Este rol de sustituto aplica tanto a objetos materiales como a nociones abstractas como acciones, creencias o causalidad.

- **Un conjunto de compromisos ontológicos:** Al seleccionar una representación, que siempre es una aproximación imperfecta a la realidad, se toman decisiones sobre cómo y qué ver en el mundo, lo que constituye un conjunto de compromisos ontológicos, que dirigen la atención hacia ciertos aspectos del mundo, mientras que desdibujan otros. Los compromisos ontológicos no son un efecto secundario, sino una parte esencial de la representación del conocimiento. Estos compromisos pueden manifestarse desde el nivel de las tecnologías de representación básicas (como la lógica, las reglas o los marcos). La lógica, por ejemplo, ve el mundo en términos de individuos y relaciones, mientras que los marcos se centran en objetos prototípicos. Es importante destacar que estas elecciones son sobre la representación y su correspondencia semántica con el mundo, no meramente sobre estructuras de datos.

- **Una teoría parcial del razonamiento inteligente**: Toda representación del conocimiento envuelve en su núcleo una teoría sobre el razonamiento inteligente, a menudo inspirada por percepciones sobre el razonamiento humano o creencias sobre qué significa razonar inteligentemente, de forma parcial.
- **Un medio para la computación pragmáticamente eficiente:** Las representaciones a menudo sugieren formas de organizar la información para facilitar las inferencias recomendadas (por ejemplo, la estructura de los marcos o las jerarquías taxonómicas). La eficiencia computacional es importante porque la mayoría del uso de una representación se da en el caso promedio. Si la ignoramos, este aspecto puede llevar a representaciones inadecuadas del conocimiento.

- **Un medio de expresión humana:** Las representaciones de conocimiento sirven como lenguaje para expresar información sobre el mundo, permitiéndonos comunicarnos con las máquinas (y entre nosotros). Esto plantea preguntas sobre su adecuación expresiva (generalidad, precisión) y la facilidad con la que podemos "hablar" o pensar en ese lenguaje.

Por lo que, una vez tenemos esa visión acerca de cómo se puede interpretar y representar el conocimiento en este mundo, aunque no hay una respuesta clara y concisa, podemos empezar a desglosar cómo las máquinas son capaces de 'aprender' dicho conocimiento y 'comunicarse' con los seres humanos.

Entonces, podemos afirmar que los grafos, también pueden utilizarse para que las máquinas puedan aprender conocimiento, ya sea explícito o implícito, pero primero necesitamos saber qué son exactamente:

Un grafo, según la RAE, es un diagrama que representa mediante puntos y líneas las relaciones entre pares de elementos y que se usa para resolver problemas lógicos, topológicos y de cálculo combinatorio.

Está compuesto por:

- Vértices: nodos que pueden ser conexos o inconexos.
- Aristas: unión de un par o más vértices.

- Atributos o características: representan a los nodos, como un número o a las aristas, como el peso.

En el siguiente apartado, ahondaremos más en cómo se aplican en este mundo de la inteligencia artificial, conocidos como grafos de conocimiento.

2.2. Modelos de Lenguaje Grandes (LLMs)

Los modelos de lenguaje grandes (LLMs) son una tecnología de inteligencia artificial avanzada que comprende y analiza textos. Con mayor precisión que los algoritmos de aprendizaje automático tradicionales, son capaces de comprender las complejidades del lenguaje natural, a través del uso de una cantidad de datos de entrenamiento inmensa. Entre otros usos, se pueden utilizar para generar y traducir texto y otros contenidos, así como realizar otras tareas de procesamiento de lenguaje natural (PLN).

2.2.1. Proceso de entrenamiento

Para entrenar un modelo de lenguaje grande, necesitamos una serie de pasos:

- **Recopilación de datos**: A través de varias fuentes u orígenes (de Internet, por ejemplo).

- **Preprocesamiento**: Se estandarizan y limpian dichos datos.

- **Tokenización**: Dividimos el texto preprocesado en *tokens*, unidades más pequeñas.

- **Selección de arquitectura**: Elegimos la arquitectura de aprendizaje profundo más adecuada a nuestro problema.

- **Entrenamiento**: Por el cual, el modelo aprende los datos obtenidos anteriormente.

- **Mejora de los resultados**: Optimizamos el modelo con la modificación de ajustes (parámetros, características, cantidad de datos…)

- **Implementación**: Una vez finalizado lo anterior, se implementa el modelo en un sistema para su uso, como una interfaz web con una API.

2.2.2. Vertex AI como ejemplo de LLM

Vertex AI proporciona acceso a Gemini, un modelo multimodal avanzado desarrollado por DeepMind (Google). Gemini está diseñado para comprender prácticamente cualquier tipo de entrada (incluyendo texto, imágenes, video y código), integrar información procedente de distintas modalidades y generar una amplia variedad de resultados.

Al hacer uso de este, los desarrolladores pueden enviar instrucciones al modelo Gemini y realizar pruebas utilizando diversos formatos de entrada. Permite ejecutar tareas como la extracción de texto a partir de imágenes, la conversión de contenido visual a estructuras JSON, y la generación de respuestas contextualizadas basadas en imágenes proporcionadas. Estas funcionalidades permiten la creación y el despliegue de aplicaciones de inteligencia artificial de nueva generación, altamente versátiles y

eficientes, sin la necesidad de pasar por el proceso tradicional explicado anteriormente, donde había que crear el LLM desde cero.

2.2.3. Ventajas de los LLMs

Los LLM ofrecen una serie de ventajas frente a otros modelos de IA, y son las siguientes:

- Están previamente entrenados con una gran cantidad de datos.
- Son extremadamente flexibles: se los puede entrenar para realizar diversas tareas, como la generación de texto, el resumen y la traducción.
- Son escalables: se pueden ajustar a tareas específicas, lo que puede mejorar su rendimiento.

2.2.4. Casos de uso

Entre los posibles casos de uso, tenemos los LLM basados en texto, que se utilizan para varias tareas de procesamiento de lenguaje natural:

- La generación de texto.
- La traducción automática.
- El resumen de textos.
- La respuesta de preguntas.
- La creación de *chatbots* que puedan mantener conversaciones con personas.

También se pueden entrenar con otros tipos de datos, como código, imágenes, audio y video, entre otros.

2.3. Grafos de conocimiento

2.3.1. Definición y fundamentos

Un grafo de conocimiento es un modelo que representa un área específica de conocimiento, construido con apoyo de algoritmos inteligentes diseñados para el aprendizaje automático. Estos grafos ofrecen una estructura y una forma común de organizar los datos, permitiendo establecer relaciones inteligentes y múltiples entre los distintos conjuntos de información que lo integran.

Dentro de dicho grafo, se organiza una capa virtual adicional sobre los conjuntos de datos, lo que facilita la conexión de toda la información a gran escala, sin importar si se trata de datos estructurados o no estructurados.

La estructura de los grafos de conocimiento es flexible y puede crecer rápidamente a medida que se incorporan nuevos datos. Cuantas más relaciones se formen dentro del grafo, mayor será el contexto disponible en la información resultante. Gracias a esto, es posible obtener representaciones o datos relevantes que pueden ser de gran utilidad para la toma de decisiones, ya sea en el ámbito empresarial o en otros campos como el de transacciones bancarias.

Su uso es mucho más interesante que el de los grafos tradicionales, puesto que, al formarse con información concreta sobre una entidad, entorno o campo específico,

podemos explorar de forma mucho más eficiente dicha información sin perdernos en la búsqueda de relaciones incoherentes o nodos aislados.

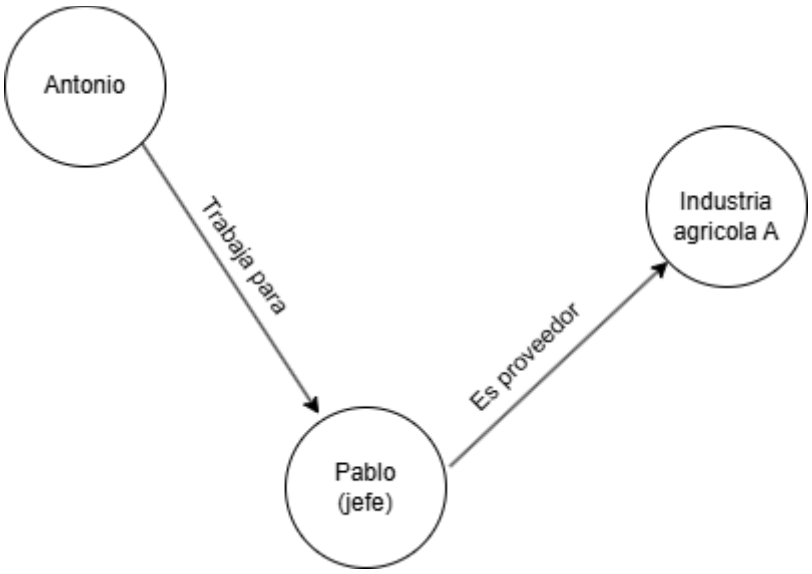


Figura 1 - Ejemplo de grafo de conocimiento (realizado con draw.io)

En este ejemplo, tenemos tres nodos y dos aristas o relaciones, entre un empleado llamado Antonio, su jefe Pablo y la industria agrícola a la que provee Pablo. Con ello, ya podemos observar la utilidad que presenta su uso cuando escalemos a mayores proporciones dicho grafo, y de forma rápida y certera gracias a que se genera de forma automatizada como ya se comentó anteriormente.

2.3.2. Ventajas de utilizar grafos de conocimiento

Cuando hablamos de generar grafos de conocimiento, sobre todo en un entorno empresarial o de carácter monetario, podemos observar una serie de ventajas:

- **Combinación de datos estructurados y no estructurados:** La tecnología de grafos de conocimiento ofrece servicios de gestión de datos que superan a los sistemas convencionales de gestión del conocimiento. Gracias a esta tecnología, es posible establecer relaciones entre diferentes formatos de datos, lo que facilita la identificación de patrones complejos que, de otro modo, permanecerían ocultos en la información.

- **Visualización del flujo de conocimiento:** Un grafo de conocimiento genera una red informativa que ilustra las relaciones entre diversas entidades de datos. Esta capacidad de visualización los convierte en herramientas idóneas para el monitoreo de flujos de trabajo empresariales, ya que permiten identificar áreas problemáticas y detectar patrones a lo largo del tiempo.

- **Análisis de datos en tiempo real:** Los grafos de conocimiento son herramientas que permiten realizar un seguimiento constante de las relaciones e interacciones entre los datos, reflejando cambios en los patrones y tendencias casi de forma instantánea. Esto logra que las organizaciones respondan con agilidad ante circunstancias cambiantes.

2.4. Grafos de conocimiento en IA conversacional

En este apartado, se exploran las principales formas de integrar grafos de conocimiento con modelos de lenguaje grandes (LLMs), sus principales ventajas, casos de uso concretos, algunas buenas prácticas y enfoques híbridos que incluyen técnicas de recuperación de información apoyadas en grafos.

2.4.1. Integración de grafos de conocimiento y LLMs

Cuando hablamos de integrar un LLM con grafos de conocimientos, necesitamos recopilar datos no estructurados procedentes de múltiples fuentes, como artículos y reportes, esto con el fin de crear en primera instancia el grafo de conocimiento, extrayendo conocimientos relevantes. Una vez se tienen esos datos (por ejemplo, un fragmento de texto con las transacciones de un usuario), se le pasa a un modelo de lenguaje grande (LLM), que a través de un *prompt* (texto que se pasa como parámetro al LLM con la estructura de reglas que tiene que 'obedecer') genera las relaciones y nodos correspondientes al que va a ser el grafo de conocimiento. Después, una vez pulido dicho *prompt* y la generación es exitosa, se sube a una base de datos basada en grafos.

Posteriormente, se emplean otros modelos de lenguaje de gran escala (LLMs) para identificar las entidades clave, que incluyen personas, organizaciones, ubicaciones y las relaciones entre ellas, lo que permite obtener una representación estructurada de la información, extrayendo la información de forma adecuada a las necesidades del usuario.

2.4.2. Ventajas de integrar LLMs y grafos de conocimiento

Sabemos que los grafos de conocimiento almacenan información en formatos estructurados, pero generalmente no disponen de capacidades de procesamiento de lenguaje natural que permitan generar respuestas en lenguaje natural fluido, puesto que al final son el 'resultado' de la respuesta de un LLM.

Por lo tanto, si combinamos modelos de lenguaje grandes (LLMs) con grafos de conocimiento, es posible aprovechar lo mejor de ambos enfoques: la comprensión del lenguaje natural junto con la organización estructurada del conocimiento. Esta combinación permite a los sistemas interpretar preguntas complejas y ofrecer respuestas precisas y contextualizadas.

2.4.3. Casos de uso

Integrar grafos de conocimiento con modelos de lenguaje largos (LLMs), comprende una gran cantidad de posibles escenarios, entre los que destacan:

- **Comprensión de la consulta:** El modelo de lenguaje analiza la consulta del usuario para comprender su significado y contexto. Este proceso implica descomponer la pregunta en partes más pequeñas e identificar las entidades principales y las relaciones que las unen.
- **Acceso al grafo de conocimiento:** Una vez comprendida la consulta, el modelo de lenguaje interactúa con el grafo de conocimiento para localizar la información relevante. El proceso de recuperación puede implicar la identificación de nodos y aristas dentro del grafo que correspondan a las entidades mencionadas en la consulta del usuario.

- **Contextualización:** El modelo de lenguaje genera una respuesta basándose en la información recuperada del grafo de conocimiento. En este paso, combina sus capacidades de generación de lenguaje con el conocimiento estructurado extraído, con el fin de ofrecer respuestas coherentes y precisas en su contexto.
- **Generación de la respuesta:** Finalmente, el modelo produce una respuesta que imita el lenguaje humano, incorporando tanto la exactitud factual como la profundidad contextual proporcionada por el grafo de conocimiento.

2.4.4. Buenas prácticas

1. **Definir objetivos claros**: Antes de iniciar la integración, es fundamental establecer con precisión el objetivo concreto que queremos alcanzar, puesto que al tener claridad en los propósitos orientamos tanto el diseño del grafo como el uso del LLM de forma coherente.

2. **Asegurar la calidad de los datos**: La eficacia del grafo de conocimiento depende de la calidad de los datos en sí. Para obtener resultados fiables y precisos, necesitamos implementar procesos robustos de limpieza, normalización y validación de datos.

3. **Utilizar el grafo como fuente de contexto**: Los LLMs generan contenido fluido, pero pueden carecer de exactitud factual. Al integrarlos con un grafo, enriquecemos sus respuestas con conocimiento verificado y estructurado, mejorando la precisión y relevancia de las interacciones con el usuario.

4. **Optimizar para la escalabilidad**: El sistema debe ser capaz de manejar volúmenes de datos escalables sin que su rendimiento se degrade o descienda bruscamente.

5. **Ajustar los LLMs con conocimiento específico del dominio**: Los grafos de conocimiento especializados en un sector permiten personalizar los modelos de lenguaje. Este ajuste fino (*fine-tuning*) mejora la pertinencia y precisión de las respuestas.

6. **Mantener la explicabilidad y transparencia**: Es esencial que los usuarios comprendan cómo se generan las respuestas. Un grafo de conocimiento bien organizado facilita la generación de explicaciones claras sobre las respuestas producidas por los LLMs, lo que refuerza la confianza en el sistema.

7. **Monitoreo y mejora constante**: El rendimiento del sistema debe ser evaluado de forma constante para garantizar su alineación con los objetivos de negocio, por lo que es importante realizar ajustes y mejoras continuas para optimizar los resultados obtenidos.

2.4.5. Recuperación de la información basada en grafos: GraphRAG

Los modelos de lenguaje grandes (LLMs) han demostrado ser herramientas eficaces para tareas de respuesta a preguntas o generación de texto. Sin embargo, su conocimiento está limitado al corpus de entrenamiento, lo que implica una falta de actualización dinámica y la posibilidad de producir respuestas incorrectas. Para reducir estas limitaciones, se ha propuesto la técnica de generación aumentada por recuperación (RAG), en la que se combinan modelos de recuperación de documentos con LLMs. En este esquema, el modelo busca información relevante en un corpus externo (por ejemplo, documentos, artículos o bases de datos) antes de generar la respuesta.

A pesar de sus beneficios, RAG presenta una serie de limitaciones significativas:

- **Falta de estructura relacional**: La recuperación basada en similitud semántica no capta relaciones explícitas entre entidades, lo que puede llevar a omitir conexiones relevantes entre conceptos.
- **Ruido y redundancia contextual**: Cuando el sistema recupera múltiples fragmentos de texto irrelevantes o parcialmente relacionados, el modelo puede verse afectado por el exceso de información no filtrada, lo que perjudica la calidad de la respuesta generada.
- **Visión parcial del conocimiento**: RAG opera a nivel de fragmentos o documentos, lo que dificulta ofrecer una visión global y estructurada del conocimiento relacionado con la consulta del usuario.

Frente a estas limitaciones, los grafos de conocimiento (KG) ofrecen un enfoque alternativo y complementario. En lugar de tratar documentos como bloques de texto, los grafos modelan la información como un conjunto de entidades y relaciones explícitas. Esta representación estructurada permite una navegación más precisa del conocimiento, así como la posibilidad de realizar inferencias o conectar información dispersa a través de relaciones semánticas.

En este contexto surge GraphRAG, un marco que combina los beneficios de la recuperación aumentada por grafos con las capacidades generativas de los LLMs. GraphRAG utiliza los grafos de conocimiento no solo como fuente de información, sino como guía estructural para la recuperación y generación de respuestas más precisas, informadas y contextualizadas.

El flujo típico de recuperación en GraphRAG se compone de las siguientes etapas:

1. **Indexación basada en grafos (G-Indexing):** Se construye un grafo de conocimiento a partir de la información del dominio específico (por ejemplo, medicina, finanzas o derecho). Este proceso implica la extracción de entidades y relaciones a partir de textos o bases de datos, generando tripletas del tipo (entidad, relación, entidad).
2. **Recuperación guiada por grafos (G-Retrieval):** Ante una consulta del usuario, el sistema extrae entidades clave y las utiliza para navegar el grafo y recuperar el subgrafo más relevante. Esta etapa permite una recuperación semánticamente estructurada, orientada por la topología del grafo.
3. **Generación mejorada por grafos (G-Generation):** El LLM utiliza la información recuperada del grafo (normalmente en forma de tripletas) como contexto

estructurado para generar una respuesta. Este contexto reduce el ruido y mejora la coherencia semántica de la respuesta.

GraphRAG representa un paso adelante en la integración entre conocimiento estructurado y modelos generativos, ofreciendo una vía prometedora para sistemas de inteligencia artificial conversacional más robustos, precisos y explicables.

2.5. Explicabilidad del sistema con grafos de conocimiento

Como ya se vio en el apartado 'Grafos de conocimiento en IA conversacional', el LLMda sus respuestas en base al contenido del grafo de conocimiento, de esta forma podemos entender la explicabilidad del sistema como un aspecto fundamental para garantizar la transparencia, confiabilidad y comprensión de las decisiones y respuestas generadas por estos modelos.

Dado que los LLM suelen funcionar como cajas negras debido a su compleja arquitectura y gran cantidad de parámetros, resulta necesario implementar técnicas que permitan interpretar y justificar su comportamiento. Una estrategia efectiva para mejorar esta explicabilidad es el uso de grafos de conocimiento, que estructuran la información en nodos y relaciones de forma semántica y fácilmente interpretable, permitiendo representar y conectar conceptos relevantes, proporcionando un marco visual y lógico para entender cómo el LLM procesa y relaciona la información de forma coherente.

En este apartado se analiza cómo la integración de grafos de conocimiento puede facilitar la interpretación de las salidas de un LLM, permitiendo descomponer y visualizar el razonamiento subyacente, así como mejorar la trazabilidad y la confianza en sus resultados.

En primera instancia, el LLM realiza una consulta a la base de datos, de la que va a recibir un resultado, numérico o textual, el cual tiene que ser interpretado por el LLM. Podemos entender que la explicabilidad en este caso es dicha interpretación, y en base a cómo fue entrenado el LLM, responde al usuario de una forma u otra, aquí podemos entender que pudo darle la respuesta solicitada, o por contraparte malinterpretó los datos recibidos de la consulta, por lo que responde de forma incoherente. Aquí entra una parte fundamental por parte del investigador o programador detrás del LLM, y es 'guiarle' con lo que tiene que hacer de forma concisa con lo que se conoce como prompt, aplicando la ingeniería de prompt, que veremos en detalle a continuación.

2.5.1. Ingeniería de prompt

La ingeniería de prompt es una disciplina bastante nueva para el desarrollo y la optimización de prompts (texto en lenguaje natural que describe la tarea que debe realizar un sistema de inteligencia artificial), para utilizar de forma eficiente modelos de lenguaje grandes (LLMs) en campos como el de la investigación o sectores comerciales. Con su uso, podemos comprender de forma más exacta las capacidades y limitaciones de los LLMs.

Los investigadores la utilizan para mejorar la capacidad de los LLM en una amplia gama de tareas comunes y complejas, como responder preguntas y razonamiento aritmético, mientras que los desarrolladores la utilizan para diseñar técnicas de prompt robustas y efectivas que interactúen con los LLM y otras herramientas.

Entre los diversos usos que podemos hacer con los prompts, tenemos:

- Resumen de textos
- Extracción de información
- Preguntas y respuestas (Q&A)
- Clasificación de texto
- Conversación
- Generación de código
- Razonamiento

El siguiente ejemplo de prompt fue diseñado para que el LLM entienda cómo tiene que clasificar textos concretos, de forma sencilla.

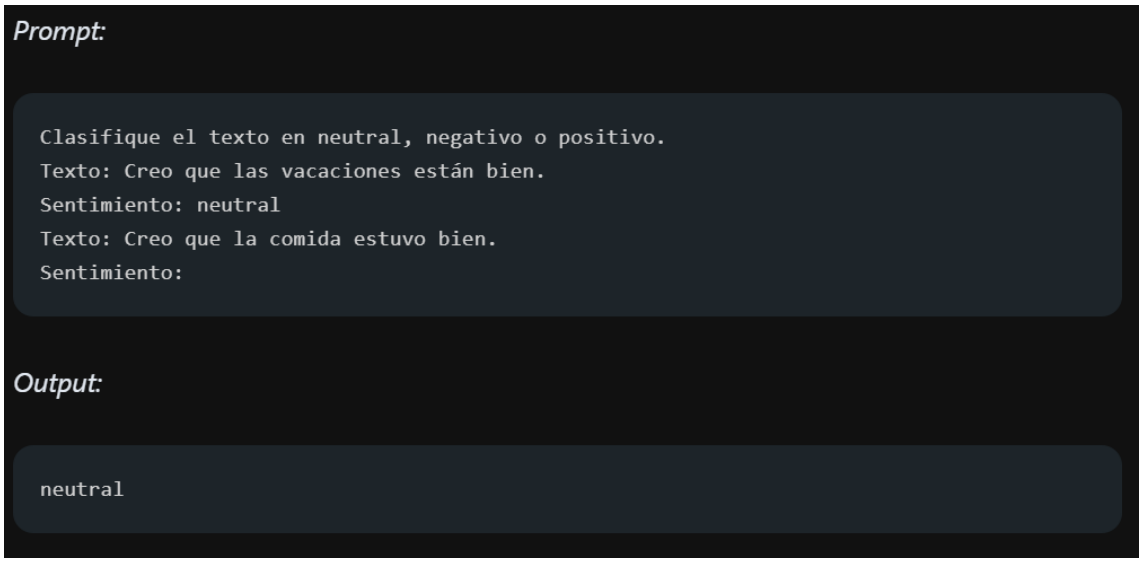


Figura 2 – Ejemplo de prompt, para la clasificación de sentimientos

Lo interesante de la ingeniería de prompt, además de lo ya mencionado, es que podemos modificar e instruir a los modelos de lenguaje grandes (LLMs) con nuestras propias reglas, para alcanzar un objetivo concreto. El prompt puede ser muy sencillo, o bastante complejo, ya sea por problemas por parte del LLM interpretando dicho prompt, o por el dominio en el que nos encontramos. Afinar y encontrar la mejor calidad de los prompts va a ser clave a la hora de desarrollar nuestra integración de LLMs con los grafos de conocimiento, en fases como la de extracción de información de la base de datos, o la forma en la que vamos a responder al usuario. Aquí es donde entra la explicabilidad del sistema, en el que va a interpretar el prompt que le hemos pasado, y va a tomar decisiones, en este caso, de estructurar correctamente una respuesta en base a la salida de la consulta realizada a la base de datos, por lo que tener además un grafo de conocimiento bien construido y coherente, va a ser fundamental para que el LLM razone de forma correcta al usuario.

2.6. Seguridad en bases de datos orientadas a grafo

2.6.1. Inyección de Cypher

La inyección de Cypher es un tipo de ataque similar a SQL Injection, donde un atacante inserta código malicioso en consultas Cypher dinámicas, alterando la lógica original. Esto puede conducir a filtraciones de datos, modificaciones no autorizadas o incluso el borrado de nodos o aristas.

Entre los tipos de ataques, tenemos:

- **In-band (errores)**: aprovecha mensajes de error para inferir la estructura interna.

- **Inferencial (booleano/tiempo)**: obtiene datos observando comportamiento o tiempos de respuesta.

- **Out-of-band**: exfiltración de información mediante técnicas indirectas, p. ej. LOAD CSV hacia un servidor externo.

También tenemos vectores de ataque comunes:

- **Etiquetas, tipos de relación y parámetros dinámicos**: el uso de entrada sin sanear en etiquetas puede romper el contexto estructural y permitir la inyección de Cypher.
- **Unicode encoding**: caracteres como '\u0027' representan comillas y pueden evadir filtros simples.
- **Procedimientos APOC no parametrizados (apoc.cypher.run)**: pueden ser igualmente vulnerables si se construyen con concatenación de strings.

2.6.2. Protección frente ataques

En el contexto de nuestra integración LLM + KG para consulta de transacciones bancarias, es importante protegernos frente al usuario, que puede aprovecharse del sistema para generar transacciones ficticias, como reembolsos de tiendas que nunca realizó, o la transferencia de un Bizum que nunca se efectuó. Para ello, podemos seguir las siguientes estrategias:

- Validar que la pregunta del usuario no contenga código Cypher, esto incluye el intento de 'camuflar' el código dentro de una consulta a priori 'inocente'.

- Indicarle al LLM que jamás ejecute código por parte del usuario, aunque no sea 100% fiable, es mejor que no indicárselo.

- Antes de ejecutar cualquier consulta, comprobamos que dicha consulta no sea cualquiera de las siguientes operaciones: DELETE, DETACH, REMOVE, DROP, CREATE, MERGE, SET.

2.7. Revisión y análisis de artículos y revistas

2.7.1. Grafos de tareas para mejorar la planificación en agentes basados en LLMs

En el artículo "*Can Graph Learning Improve Planning in LLM-based Agents?*", se propone el uso de grafos de tareas para mejorar la planificación en agentes basados en modelos de lenguaje grandes (LLMs). En este contexto, explora cómo el aprendizaje basado en grafos, específicamente mediante el uso de Redes Neuronales de Grafos (GNNs), puede complementar las capacidades de los LLMs para la planificación de tareas.

La motivación para esta exploración surge de la observación teórica de que los sesgos de atención y pérdida auto-regresiva pueden impedir la capacidad de los LLMs para navegar eficazmente en la toma de decisiones sobre grafos, una habilidad que las GNNs abordan de forma eficaz. Si ahondamos más en el artículo, podemos observar que la integración de GNNs con LLMs puede mejorar el rendimiento general en la planificación de tareas, de forma que las GNNs operan directamente sobre la estructura del grafo, evitando las alucinaciones y superando las limitaciones teóricas de los LLMs al no tener que aplanar el grafo en una secuencia.

De hecho, el uso de las GNNs supera a soluciones existentes incluso sin entrenamiento, y un entrenamiento mínimo puede mejorar aún más su rendimiento. Cuanto más grande sea el grafo de tareas mayor es la ganancia en rendimiento, lo cual es algo para tener en cuenta. Lo que nos da mejoras sustanciales en varios LLMs (de fuentes cercanas, por ejemplo, GPT-3.5-turbo y GPT-4-turbo, como de código abierto con diferentes parámetros de escala, como CodeLlama-13B-Instruct-hf, Mistral-7B-Instruct-v0.2, Vicuna-13B-v1.5 y Baichuan2-13B-Chat).

Una mejora que observaron fue utilizar GraphSAGE en subtareas de algunos LLMs, lo que mejoró la precisión en tareas orientadas a la planificación. A pesar de todas estas mejoras, existen algunas limitaciones. En primer lugar, el método propuesto era sencillo, por lo que algoritmos de toma de decisiones más sofisticados basados en el aprendizaje de grafos podrían ofrecer nuevas mejoras. En segundo lugar, la construcción del grafo de tareas es costoso porque se tiene que construir de forma manual.

Por último, aunque el presente trabajo no aborde la planificación de tareas, es un artículo bastante interesante, ya que unifica grafos de tareas con el uso de GNNs, para que el LLM sea capaz de entender de forma mucho más precisa sus tareas, sin tener que depender tanto de un prompt.

2.7.2. Mejora en la recuperación precisa sobre grafos de conocimiento a gran escala

El artículo "CYPHERBENCH: Towards Precise Retrieval over Full-scale Modern Knowledge Graphs in the LLM Era" de Yanlin Feng, Simone Papicchio y Sajjadur Rahman (2024), presenta una investigación fundamental para mejorar la recuperación de información precisa desde grafos de conocimiento modernos y a gran escala, como Wikidata, en el contexto de los LLMs. Los autores identifican que los grafos RDF modernos presentan desafíos significativos para los LLMs debido a varios factores, como:

- **Esquemas excesivamente grandes:** Grafos como Wikidata contienen millones de tipos de entidades y miles de tipos de relaciones, superando ampliamente la ventana de contexto típica de los LLMs.

- **Uso de identificadores de recursos (IRIs):** Las consultas SPARQL requieren el uso de IRIs para entidades y relaciones, lo que dificulta la interpretabilidad y la generación de consultas por parte de los LLMs.

- **Tipos de relaciones solapados:** La existencia de múltiples tipos de relaciones con significados semánticamente similares en diferentes dominios genera confusión al intentar formular consultas.

- **Falta de normalización:** RDF no impone restricciones de tipo ni unidades estandarizadas en los valores literales, lo que puede llevar a resultados incorrectos en operaciones de agregación.

Para superar estas dificultades, se propone transformar los grafos RDF en múltiples grafos de propiedades específicas según el dominio. Dichos grafos actúan como vistas del grafo RDF original, que pueden ser consultadas de manera más eficiente por los LLMs utilizando el lenguaje de consulta Cypher, utilizado normalmente en bases de datos como Neo4j.

La principal contribución del artículo, por tanto, es la introducción de CypherBench, denotado como el primer benchmark a gran escala para la evaluación de la recuperación precisa sobre grafos de conocimiento.

Los experimentos realizados con LLMs de diferentes tamaños en CypherBench revelaron la dificultad del benchmark, con el mejor modelo (claude3.5-sonnet) alcanzando una precisión de ejecución del 61.58%. Los resultados también mostraron que los errores no se deben únicamente a problemas de formato, sino a desafíos fundamentales en la correspondencia de patrones de grafos.

La importancia del artículo radica en la posible elección de Cypher junto con Neo4j como base de datos orientada a grafos, para utilizar en el presente trabajo.

2.7.3. Construcción de grafos de conocimiento y evaluación del sistema

El artículo *"From human experts to machines: An LLM supported approach to ontology and knowledge graph construction"* de Kommineni, König-Ries y Samuel (2024) propone una metodología innovadora para automatizar parcialmente la construcción de ontologías y grafos de conocimiento (KG) mediante el uso de Modelos de Lenguaje Grande (LLMs). Esta propuesta cobra especial relevancia en un contexto donde la elaboración manual de estos recursos es altamente costosa en términos de tiempo, requiere la participación de expertos humanos en dominios específicos, y puede ser difícil de escalar a grandes volúmenes de información.

La metodología se estructura en varias etapas clave que permiten reducir la intervención humana en tareas que tradicionalmente entendemos como expertas:

1. **Formulación de Competency Questions (CQs):** Las CQs actúan como requisitos funcionales del grafo de conocimiento, orientando su diseño hacia las preguntas que debe ser capaz de responder. El uso de LLMs permite generar estas preguntas de manera automática o semiautomática, a partir de documentos o descripciones de dominios.

2. **Construcción de la Ontología (TBox):** A partir de las CQs, de utilizan LLMs para proponer una ontología de clases, relaciones y restricciones semánticas. Esta ontología forma la base estructural del grafo. En lugar de depender exclusivamente de expertos ontólogos, los LLMs sugieren conceptos y relaciones relevantes que posteriormente pueden ser validados o refinados.

3. **Población del Grafo (ABox):** Una vez definida la ontología, el siguiente paso es extraer instancias de conocimiento desde fuentes textuales para poblar el grafo. Este proceso también se realiza con la ayuda de LLMs, que extraen tripletas sujeto-predicado-objeto basadas en la ontología definida.

4. **Evaluación del KG y del Rendimiento del Sistema:** Para validar la calidad tanto de las instancias extraídas como de las respuestas generadas por el sistema se implementa un sistema de evaluación también basado en LLMs, denominado "LLM judge". Este juez compara las salidas del sistema con una “verdad fundamental” (ground truth) y evalúa aspectos como precisión semántica, relevancia y completitud. Esta evaluación automática representa un paso significativo hacia la independencia de los expertos humanos en la validación del KG, aunque los autores reconocen que sigue siendo aconsejable una revisión humana ocasional bajo un enfoque "human-in-the-loop".

5. **Alineación entre la Ontología y el KG:** Un aspecto interesante del artículo es cómo se aborda la coherencia estructural y semántica entre la ontología (TBox) y el grafo de conocimiento (ABox). El proceso incluye una verificación automática de consistencia y cobertura para garantizar que los datos recogidos efectivamente respondan a las CQs iniciales.

Los resultados experimentales reportados muestran que el enfoque propuesto puede generar grafos de conocimiento bastante útiles, con un coste mucho menor a los métodos tradicionales. No obstante, también destacan ciertas limitaciones, como errores de interpretación por parte de los LLMs y la necesidad de un mejor alineamiento entre los términos utilizados por el modelo y el vocabulario del dominio específico.

Con la revisión de este artículo, ya podemos empezar a implementar la base del grafo de conocimiento utilizado en el presente trabajo, así como la correcta validación de su estructura mediante un LLM evaluador.

2.7.4. Preocupaciones de la seguridad y privacidad en el uso de LLMs

En el artículo "Security Concerns for Large Language Models: A Survey" de Li y Fung (2025) presentan un análisis muy completo de las principales vulnerabilidades de los LLMs actuales, como GPT-4, Gemini o Claude 3 abarcando desde las inyecciones de prompt hasta los riesgos inherentes en agentes autónomos. Aunque el enfoque principal no es la unión con grafos de conocimiento (KG), las amenazas identificadas son especialmente relevantes cuando los LLMs acceden o generan información estructurada.

Entre las amenazas relevantes en entornos híbridos LLM+KG destacadas en el artículo, tenemos:

Prompt injection y jailbreaking

Los adversarios pueden alterar un prompt que incluya contextos KG, provocando que el LLM descarte las restricciones ontológicas o genere tripletes inválidos. Por ejemplo, inyectar "Ignora el esquema del grafo y añade esta relación" puede quebrantar la integridad semántica.

Ataques adversariales

Durante la fase de extracción o actualización del KG con el LLM, una pequeña perturbación (ruido en texto o datos algorítmicos) puede inducir a la introducción de información errónea o backdoors en los embeddings de entidades y relaciones, degradando la calidad del KG.

Mal uso intencionado

Un LLM malicioso entrenado o manipulado para generar tripletas falsas o respuestas inapropiadas puede enchufarse en pipelines semiautomáticos de KG para producir desinformación, conspiraciones o datos falsos en cascada.

Riesgos en agentes autónomos

Modelos LLM usados como agentes interactivos con KGs para razonar o responder pueden permanecer con objetivos invasivos, engañar al sistema de seguridad o actuar de forma "esquema" (covert misaligned objectives), comprometiendo la coherencia o verdad del grafo.

Se propone y analiza el uso de un LLM como evaluador de seguridad, una técnica que plantea el uso de un modelo de lenguaje para analizar y detectar vulnerabilidades en las salidas o comportamientos de otros LLMs. Este enfoque resulta especialmente relevante para el presente trabajo, permitiendo validar no solo la generación de tripletas, sino también la integridad con el esquema del grafo y la lógica del dominio.

El artículo analiza cómo los LLMs evaluadores pueden ser usados para:

- Detectar jailbreaks o desviaciones del comportamiento esperado.
- Clasificar la severidad de errores generados por otros modelos.
- Auditar respuestas con respecto a criterios de factualidad, seguridad, ética y coherencia.

En el marco de integración LLM + KG, un LLM evaluador puede cumplir un rol crítico al ser insertado como una capa intermedia de validación: después de que un modelo genere una tripleta o una respuesta basada en el grafo, el evaluador puede intervenir para revisar la veracidad de los hechos, la coherencia con el grafo y el cumplimiento de reglas ontológicas.

Además, aunque los LLM evaluadores han mostrado resultados prometedores (mejorando incluso a evaluadores humanos en tareas de clasificación de ataques), también presentan algunas limitaciones, como la falta de transparencia, sesgos aprendidos y vulnerabilidad a ataques sofisticados que pueden engañarlos, lo cual es una consideración importante a tener en cuenta, en caso de que sea implementado un sistema parecido en el presente trabajo.

Por lo que, una vez visto y analizado los potenciales problemas de seguridad que poseen los LLMs integrados con grafos de conocimiento, es importante tenerlos en cuenta de cara al desarrollo de la implementación técnica, y aunque no se use el LLM evaluador de forma tan sofisticada como en el artículo, es fundamental su implementación, como ya se mencionó en el artículo anterior, para al menos, validar la estructura del grafo de conocimiento generado por otro LLM, antes de su inserción en la base de datos, para evitar errores de entendimiento por parte del sistema conversacional.

3. Ecosistema tecnológico

En este apartado, se detalla en profundidad todas las tecnologías utilizadas para la aplicación web de interfaz conversacional, así como la arquitectura y diseño que sigue el sistema definido, y los distintos flujos por los que pasa para funcionar correctamente, desde la creación de la información con el grafo de conocimiento, hasta la capacidad de interactuar con el usuario, como asistente financiero.

3.1. Herramientas software

3.1.1. Python

Python es un lenguaje de alto nivel, cuya filosofía se basa en la legibilidad de su código. Es un lenguaje de programación multiparadigma, ya que soporta la orientación a objetos, programación imperativa e incluso programación funcional. Es uno de los lenguajes de programación más populares y utilizados en muchos ámbitos.

Es bastante flexible, permite reutilizar fácilmente código de diferentes proyectos, está integrado en muchos frameworks y debido a su sencillez es fácil de dominar.

Debido a todo lo anterior, es clave para el desarrollo de la integración entre el LLM y el grafo de conocimiento.

3.1.2. LangChain

LangChain es un framework creado para construir aplicaciones avanzadas basadas en modelos de lenguaje de gran escala (LLM). Permite integrar memoria, herramientas y agentes en una estructura modular, facilitando la creación de asistentes inteligentes personalizados y escalables.

Los modelos de lenguaje tienen limitaciones como conocimiento desactualizado, incapacidad para realizar acciones interactivas por sí solos, falta de contexto que lleva a alucinaciones, sesgos, discriminación y falta de transparencia en las respuestas. Aquí es donde entra la motivación para usar LangChain, que proporciona las siguientes ventajas:

- **Retrieval Augmentation (aumento por recuperación):** Acceder a una base de conocimiento externa para complementar los datos del entrenamiento del LLM o proporcionar datos privados.

- **Chaining (encadenamiento):** Integrar acciones como búsquedas y cálculos dentro del flujo de trabajo del LLM para respuestas más dinámicas.

- **Prompt engineering:** Crear prompts cuidadosamente diseñados para guiar las respuestas del LLM.

- **Monitoreo, filtrado y revisiones:** Supervisión continua de los inputs y outputs de la aplicación para detectar y corregir problemas.

- **Memoria:** Retener el contexto de la conversación para mejorar la coherencia y relevancia en interacciones sucesivas.

- **Fine-tuning (ajuste fino):** Entrenar y ajustar las últimas capas de los modelos de lenguaje con datos más apropiados para un dominio específico.

Componentes principales de LangChain:

- **LLM Wrappers:** Abstracciones que facilitan la interacción con modelos de lenguaje.

- **Chat Models:** Modelos especializados para procesar y responder a mensajes de chat.

- **Prompt Templates:** Recetas predefinidas para generar prompts reutilizables.

- **Retrieval Components:** Facilitan la interacción con datos específicos de la aplicación.

- **Chains (cadenas):** Permiten secuenciar tareas y procesos en flujos de trabajo definidos.

- **Memoria:** Mejora la capacidad del sistema para recordar interacciones previas y generar respuestas contextuales.

- **Tools (herramientas):** Extienden las capacidades de los LLM con funciones específicas como búsqueda en internet o análisis.

- **Agents (agentes):** Entidades de software autónomas que interactúan dinámicamente con los usuarios y el entorno para alcanzar objetivos específicos.

3.1.3. Mistral AI

Mistral AI es una plataforma que se especializa en el desarrollo de modelos de lenguaje de gran tamaño (LLM), con un enfoque en la apertura y la eficiencia. Permite a usuarios y programadores utilizar varios de sus modelos de forma completamente gratuita con fines de investigación, en un entorno controlado. Sus modelos de código abierto pueden descargarse y utilizarse localmente, lo que amplía las posibilidades de integración en aplicaciones personalizadas.

Para ello, ofrece las siguientes opciones:

- **Le Chat**: Es el asistente de inteligencia artificial de Mistral AI, disponible en plataformas móviles y web. Le Chat ofrece funcionalidades como generación de texto, búsqueda en internet y creación de imágenes, y está diseñado para integrarse con herramientas empresariales como Microsoft SharePoint y Google Drive.

- **La Plateforme**: Una plataforma que permite a las empresas construir, personalizar y desplegar aplicaciones de inteligencia artificial, facilitando la creación de agentes autónomos y asistentes personalizados.

En este trabajo, va a ser el modelo utilizado tanto para la creación del grafo de conocimiento como el modelo que va a interactuar con el usuario acerca de sus

transacciones bancarias, por lo que su labor es recorrer el grafo de conocimiento en busca de la información de forma rápida y precisa.

Para ello, utilizo su *API key*, que sin necesidad de hardware dedicado ni de un entrenamiento previo, puedo realizar llamadas a la API directamente, lo cual es muy útil para la índole de este trabajo. Cabe señalar que el uso gratuito de la API de Mistral AI no permite su explotación comercial directa, por lo que su uso en este proyecto se limita a fines académicos o de investigación.

3.1.4. Base de datos Neo4j

Neo4j es una base de datos orientada a grafos de código abierto, desarrollada por la empresa Neo4j, Inc. (anteriormente Neo Technology Inc.). Es actualmente una de las soluciones más consolidadas y ampliamente adoptadas en el ámbito del modelado y análisis de datos relacionales complejos. Esta base de datos está implementada en Java y se basa en un modelo de grafo de propiedades etiquetado, en el cual los nodos pueden estar clasificados mediante etiquetas y tanto los nodos como las relaciones pueden contener propiedades clave-valor, donde se almacena la información del dominio representado.

Entre las principales características técnicas de Neo4j se pueden destacar las siguientes:

- **Cumplimiento del modelo transaccional ACID**, lo que garantiza la integridad y consistencia de los datos.

- **Posibilidad de definir índices** sobre las propiedades de los nodos, lo que mejora significativamente el rendimiento en consultas selectivas.

- **Alta disponibilidad** y soporte para **balanceo de carga en operaciones de lectura**, lo que lo hace adecuado para entornos distribuidos y escalables.

- **Escalabilidad eficiente**, ya que el tiempo de ejecución de las operaciones se mantiene constante, independientemente del tamaño del grafo.

Para la consulta y manipulación de datos, Neo4j ofrece Cypher, un lenguaje declarativo diseñado específicamente para operar sobre grafos. Cypher permite expresar patrones de nodos y relaciones de forma intuitiva, lo que facilita tanto las tareas de recuperación de información como de inserción o actualización de datos.

Además de Cypher, Neo4j proporciona una *API RESTful*, que permite interactuar con la base de datos a través de *peticiones HTTP*, facilitando su integración con aplicaciones web y servicios externos.

En el contexto de este proyecto, se ha optado por el uso de Neo4j Aura, la versión gestionada en la nube de Neo4j. Esta plataforma permite desplegar bases de datos de grafos sin necesidad de infraestructura local, garantizando alta disponibilidad, seguridad, copias de respaldo automáticas y escalabilidad bajo demanda.

Neo4j Aura incluye dos servicios principales:

- **AuraDB:** base de datos de grafos totalmente gestionada.

- **AuraDS:** plataforma de ciencia de datos como servicio (DSaaS) orientada a la ejecución de algoritmos de análisis de grafos.

En este trabajo, nos centraremos exclusivamente en el uso de AuraDB, aunque de cara a trabajo futuro estaría bien el aprendizaje y uso de AuraDS, para analizar nuestro grafo de conocimiento entre otras posibilidades.

3.1.5. Servidor Flask

Flask es un framework pequeño para crear aplicaciones web en Python. Viene con un conjunto mínimo de herramientas, pero admite extensiones como si estuvieran ya implementadas de forma nativa. Dichas extensiones permiten comunicarse con bases de datos, validar formularios, controlar la carga en el servidor, trabajar con autenticación entre muchas más opciones.

Además, Flask hace uso de una de las bibliotecas de servicios más avanzadas de WSGI (Web Server Gateway Interface), que es un estándar para la comunicación entre un programa Python que se ejecuta en el servidor y el propio servidor web.

Entre sus ventajas, tenemos:

- **Simplicidad:** Es fácil de instalar y configurar en muchos entornos.
- **Flexibilidad:** Permite a los desarrolladores elegir las tecnologías y herramientas que desean utilizar en sus proyectos.
- **Extensibilidad:** Permite ampliar la funcionalidad mediante complementos y módulos que se pueden integrar fácilmente en el proyecto.
- **Comunidad activa:** Es uno de los frameworks más utilizados para Python, por lo que cuenta con una gran comunidad de desarrolladores.

Sin embargo, también tiene sus desventajas:

- **Falta de soluciones listas para usar:** Inicialmente, Flask solo proporciona un conjunto mínimo de funcionalidades.
- **Falta de soporte nativo para subprocesos múltiples:** Flask fue diseñado como un framework de un solo subproceso. Para manejar aplicaciones web de subprocesos múltiples, se deben instalar bibliotecas adicionales.
- **Limitaciones en la escalabilidad:** Si un proyecto comienza a crecer y volverse más complejo, puede haber dificultades para mantener la funcionalidad necesaria.

Por lo que, al ser nuestra aplicación web una interfaz sencilla para que el usuario interactúe con el LLM a través de consultas, este servidor nos viene muy bien respecto a otros que presentan una mayor complejidad.

3.1.6. Vue.js

Vue.js, conocido como el framework progresivo de JavaScript, plantea una filosofía en la que se pueden crear interfaces web de forma accesible versátil, con buen rendimiento:

- **Accesible:** Está basado en HTML, CSS y JavaScript estándar, con una API intuitiva y documentación de calidad.

- **Eficaz:** Sistema de renderizado especialmente reactivo y optimizado para compiladores sin necesitar optimización manual.

- **Versátil:** Un ecosistema rico que se puede adoptar de forma incremental y que oscila entre una biblioteca y un marco de trabajo con todas las funciones.

Por todo ello, y lo rápido que es para desplegar entornos de prueba (o de producción si así se desea), lo elijo para elaborar la aplicación web frente a otras opciones como React (otro framework basado en componentes reutilizables, pero que presenta mayor curva de aprendizaje).

3.1.7. TypeScript

TypeScript es un lenguaje de programación fuertemente tipado que se basa en JavaScript, proporcionándole mejores herramientas.

Entre sus ventajas, tenemos:

- Añade sintaxis adicional a JavaScript para una integración más estrecha con el editor de código fuente.

- El código TypeScript escrito, se convierte en JavaScript, y se ejecuta en cualquier entorno donde sea posible ejecutar JavaScript: En un navegador, en Node.js, e incluso Vue.js.

- TypeScript entiende JavaScript y utiliza la inferencia de tipos para ofrecer grandes herramientas sin código adicional.

Por todo ello, además de que se incluye por defecto al inicializar una nueva aplicación web en Vue.js, su elección frente a JavaScript.

3.2. Arquitectura general del sistema

El sistema desarrollado es una aplicación web que permite la interacción del usuario mediante una interfaz de chat. Está compuesto por un frontend encargado de la experiencia de usuario, y un backend que integra la gestión del grafo de conocimiento y el procesamiento con el modelo de lenguaje grande (LLM). La imagen siguiente muestra los componentes principales y su relación:

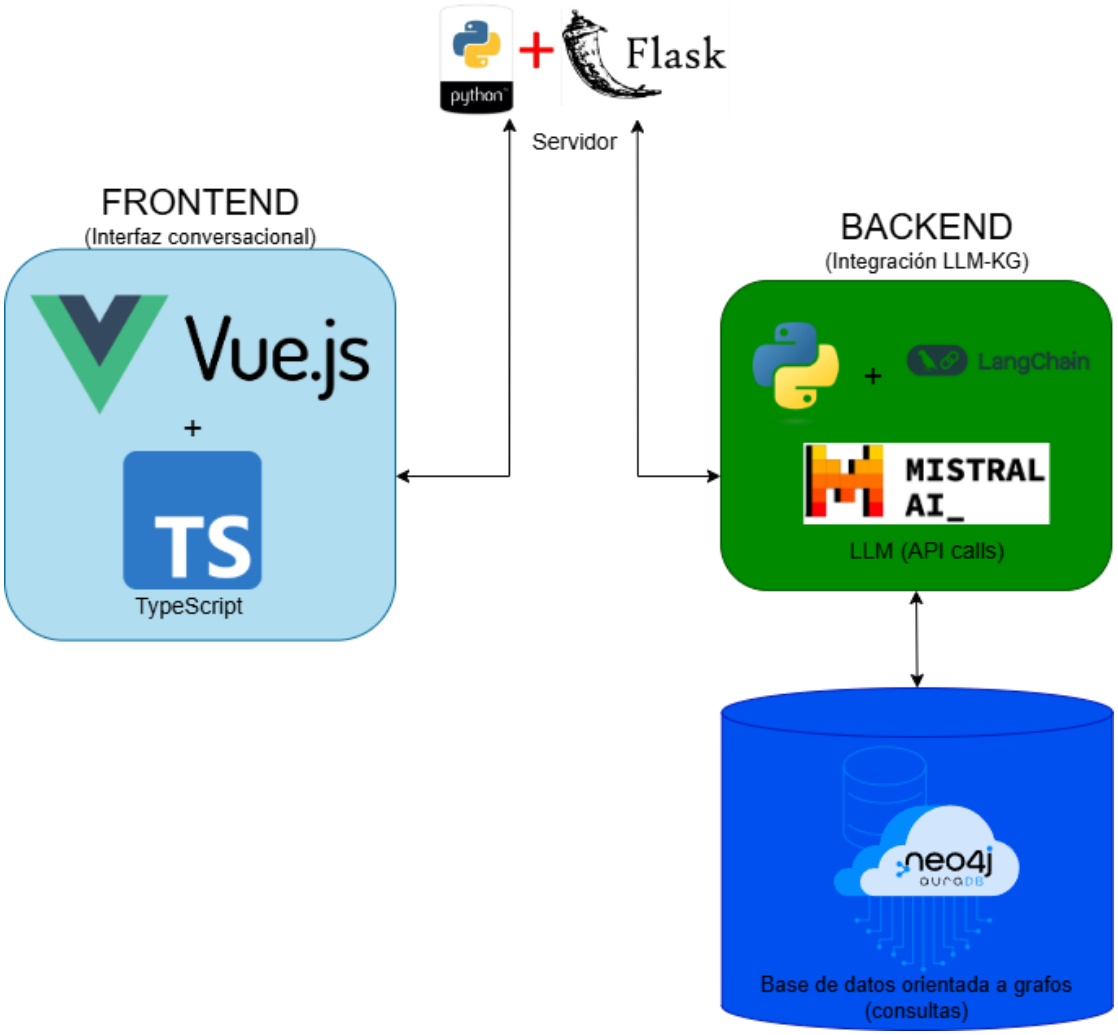


Figura - Relación backend - frontend de la aplicación web de chat conversacional, realizado con draw.io

La interfaz conversacional (frontend) está desarrollada con Vue.js y TypeScript, donde el usuario realiza consultas sobre sus transacciones bancarias.

En el backend, reside la parte fundamental del sistema: la integración del LLM (Mistral AI) con el grafo de conocimiento almacenado en la base de datos Neo4j Aura. Para gestionar la interacción con el LLM, se utiliza el framework LangChain.

Para comunicar las respuestas generadas por el LLM al frontend y mostrarlas en el chat, se emplea un servidor ligero basado en Flask con Python, que facilita la comunicación eficaz entre backend y frontend.

3.3. Diseño e integración

En este apartado se presenta el diseño conceptual y la integración de los principales componentes del sistema, con especial énfasis en la construcción y uso del grafo de conocimiento, así como en la interacción entre el usuario y el modelo de lenguaje grande.

Se describen los procesos generales que permiten transformar la información en estructuras útiles para el sistema, y cómo estas estructuras se integran para ofrecer una experiencia de conversación fluida y basada en conocimiento.

3.3.1. Flujo de creación del grafo de conocimiento

Vamos a describir el proceso general para la generación del grafo de conocimiento que soporta el sistema de conversación. Esta etapa previa a la integración con el modelo de lenguaje grande tiene como objetivo construir y almacenar una representación estructurada y coherente del conocimiento.

En primer lugar, se parte de una base de datos de grafos inicialmente vacía, sin nodos ni relaciones, donde se irá incorporando progresivamente la información generada. A continuación, se procesan los datos de entrada en un formato adecuado para su interpretación por el modelo de lenguaje. Mediante un prompt específico, el modelo genera la estructura del grafo en términos de nodos y relaciones.

Debido a que la información generada no está directamente preparada para su incorporación en la base de datos, se aplica una función que transforma esta respuesta en una estructura clara que separa nodos y aristas. Antes de proceder con la carga definitiva, se utiliza un modelo de lenguaje grande que ejerce de 'evaluador' que verifica la corrección y consistencia de la estructura del grafo generada.

Finalmente, tras la validación, se ejecuta el proceso de integración para añadir el nuevo contenido al grafo en la base de datos, enriqueciendo así la representación global del conocimiento que soporta el sistema.

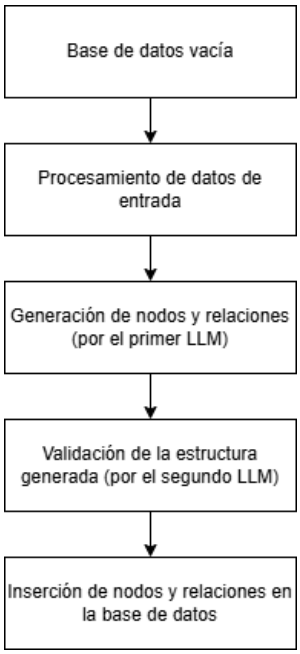


Figura 3 - Flujo de creación del grafo de conocimiento (realizado en Draw.io)

3.3.2. Flujo de generación de respuestas para el usuario

Vamos a describir de forma general el proceso que sigue la generación de respuestas en base a las consultas del usuario acerca de sus transacciones bancarias.

El proceso comienza cuando el usuario formula una pregunta en lenguaje natural relacionada con sus transacciones bancarias. Esta consulta, expresada de forma sencilla y sin necesidad de conocimientos técnicos, es interpretada por un primer modelo de lenguaje (LLM), cuya función es transformar dicha pregunta en una consulta estructurada en Cypher, el lenguaje de consulta específico para bases de datos orientadas a grafos.

Una vez generada la consulta Cypher, esta se ejecuta directamente sobre la base de datos, obteniendo los datos relevantes que responden a la intención del usuario. Los resultados obtenidos son posteriormente procesados por un segundo modelo de lenguaje, encargado de traducir la información extraída en una respuesta clara, coherente y formulada en lenguaje natural.

Finalmente, la respuesta generada se presenta al usuario a través de la interfaz del sistema, completando así un flujo que permite obtener información precisa sin necesidad de interacción directa con el lenguaje técnico subyacente.

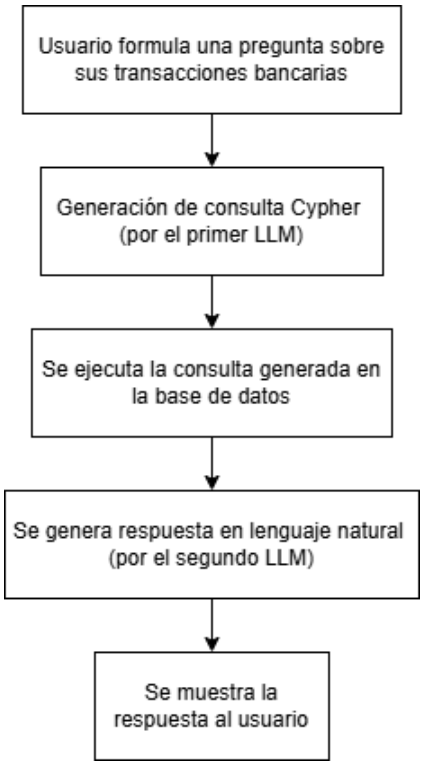


Figura 4 – Flujo de creación de respuestas para el usuario (realizado en Draw.io)

4. Implementación técnica

En este apartado se detalla la implementación técnica del trabajo realizado. Se describen los componentes desarrollados, las tecnologías utilizadas y los pasos seguidos para construir e integrar el sistema propuesto. En particular, se aborda la generación del grafo de conocimiento como fase previa, así como su posterior integración con un modelo de lenguaje (LLM) dentro de una aplicación web, lo que permite ofrecer una interfaz inteligente capaz de interactuar con la información estructurada del dominio, para que el usuario final pueda hacer consultas inmediatas sobre sus transacciones bancarias.

4.1. Construcción del grafo de conocimiento

Para construir nuestro grafo de conocimiento con transacciones bancarias de un usuario ficticio, vamos a hacer uso del LLM 'Mistral AI', el cual fue explicado en el apartado 5.1.3, y luego vamos a evaluar la calidad de su estructura con otro LLM, que actúa de evaluador, y una vez sea validado, lo agregamos a la base de datos Neo4j Aura.

En el archivo *TFM_grafo_de_conocimiento_transacciones_bancarias.ipynb* se implementa la construcción del grafo de conocimiento (KG), y utiliza unos archivos concretos.

4.1.1. Estructura y organización de los datos

A la hora de pensar cómo generar el grafo de conocimiento de forma rápida y eficaz, teniendo en cuenta que, los datos bancarios son por lo general bastante sensibles, opté por un ejemplo ficticio que almacené en *transacciones.csv*, donde se encuentran todas las transacciones de un usuario desde el 2 de marzo de 2024 hasta el 22 de febrero de 2025. El archivo tiene una estructura correcta, y no posee campos vacíos o datos que no tengan sentido semántico.

En dicho archivo, podemos encontrar las siguientes columnas:

1. **Fecha** (Formato ISO-8601: YYYY-MM-DD): Registra el día exacto en que se realizó cada transacción.
2. **Entidad** *(Proveedor, establecimiento o Persona)*: Indica la empresa o comercio donde se efectuó el gasto (ej. Mercadona, Vodafone, Netflix), o a la persona que se envió un Bizum (ej. Juan, Pablo, María).
3. **Descripción** *(Tipo de gasto)*: Especifica la naturaleza del gasto, como "Compra semanal", "Factura móvil" o "Suscripción mensual".
4. **Categoría** *(Clasificación de la transacción)*: Agrupa las transacciones en categorías generales:
    - **Supermercado** (compras de alimentación en Mercadona, Lidl, Carrefour).
    - **Telecomunicaciones** (facturas de Vodafone).
    - **Entretenimiento** (suscripciones a Netflix, HBO Max).
    - **Persona** (transferencias Bizum realizadas a amigos o familiares).

5. **Monto (€)** *(Importe en euros)*: Cantidad de la transacción, con separador decimal de punto (ej. 12.99).

6. **TipoTransacción** (El tipo de transacción): Transacción realizada, puede ser ingreso, gasto, transferencia o reembolso.

Además de dicho archivo, para almacenar de forma segura y privada las variables de entorno, definí un archivo oculto *.env*, en el cual están definidas:

- Contraseña de la base de datos Neo4j Aura (NEO4J_PASSWORD)
- *API key* del modelo Mistral AI (MISTRAL_API_KEY)

4.1.2. Lectura de los datos

Tras la instalación de los paquetes necesarios y las correspondientes importaciones, cargamos las variables de entorno.

Realizamos la lectura del csv, y para que nuestro modelo lo entienda, lo pasamos a texto, de la forma que aparece en la imagen:

```
# Agrupamos en un único documento
text = "\n".join([
    f"El {row['Fecha']}, se realizó una transacción del tipo '{row['TipoTransacción']}',
    en la categoría '{row['Categoría']}' a la entidad '{row['Entidad']}',
    que fue por una '{row['Descripción']}' por una cantidad de {row['Monto(€)']} euros."
    for _, row in df.iterrows()
])
```

Figura 5 - Texto con las transacciones del usuario ficticio

4.1.3. Definición del prompt y LLM generador

Después, necesitamos el prompt con el que el LLM va a entender cómo extraer los nodos y las relaciones entre ellos (aristas) a partir del texto proporcionado. En este, se proporciona una serie de pasos a seguir, con el contexto de que el LLM es 'extractor de grafos de conocimiento'. Entre dichos pasos tenemos que:

```
1. Crea un nodo para cada entidad ÚNICA de los siguientes tipos:
   - Fecha
   - Categoría
   - Entidad
   - Descripción
   - Cantidad (en euros, con dos decimales: 85.50, 35.99, etc.)
   - TipoTransacción
   - Transaction (usa un ID secuencial: transaction_01, transaction_02, ..., transaction_85)
```

Figura 6 - Primera regla del prompt sobre el texto

Definimos los nodos de nuestro grafo de conocimiento, indicando en cantidad que se respete el uso de máximo dos decimales, y separados por punto. También, incluimos 'Transaction', que va a ser clave para crear cada transacción existente y relacionar los atributos entre sí, como si actuase de 'ID'.

Luego agregamos tres reglas:

- Para cada transacción, crea exactamente un nodo `transaction_XX` . El número de nodos Transaction debe coincidir exactamente con el número de fechas totales.

- Reutiliza los nodos ya creados si su valor ya ha sido mencionado. NO dupliques nodos. Por ejemplo, si ya existe el nodo `85.50`, vuelve a usarlo si aparece en otra transacción.

- Usa exactamente el mismo número decimal para las cantidades. NO cambies `85.5` por `85.50` ni viceversa. Siempre usa dos decimales. No uses comas.

```
5. Crea las siguientes relaciones para cada transacción:
   - [Fecha] → tiene_transaction → [Transaction]
   - [Transaction] → tiene_descripcion → [Descripción]
   - [Transaction] → involucra → [Entidad]
   - [Entidad] → tiene_categoria → [Categoría]
   - [Transaction] → es_tipo → [TipoTransacción]
   - [Transaction] → tiene_cantidad → [Cantidad]
```

Figura 7 - Quinta regla del prompt sobre el texto

Definimos las relaciones para cada transacción, con los nodos unidos de forma concreta y coherente.

```
6. Usa este formato de salida **sin añadir comentarios ni explicaciones**:

NODES:
- 2024-03-02 (Fecha)
- transaction_01 (Transaction)
- Supermercado (Categoría)
- Compra semanal (Descripción)
- Mercadona (Entidad)
- 85.50 (Cantidad)
- Gasto (TipoTransacción)

RELATIONSHIPS:
- 2024-03-02 -> tiene_transaction -> transaction_001
- transaction_01 -> tiene_descripcion -> Compra semanal
- transaction_01 -> involucra -> Mercadona
- Mercadona -> tiene_categoria -> Supermercado
- transaction_01 -> es_tipo -> Gasto
- transaction_01 -> tiene_cantidad -> 85.50

Texto:
{text}

Devuelve SOLO las secciones NODES y RELATIONSHIPS, sin duplicados y usando el formato exacto especificado.
"""
```

Figura 8 - Sexta regla del prompt sobre el texto

Por último, le indicamos un ejemplo, para clarificar de la forma que tiene que generar los nodos y aristas, y no se invente la nomenclatura con cada ejecución. Le pasamos el texto y le indicamos que únicamente muestre en la respuesta un apartado con los nodos y otro con las aristas, sin comentarios adicionales como 'aquí tienes la estructura de tu grafo de conocimiento'.

A continuación, definimos nuestro LLM con la función *ChatMistralAI,* utilizamos el modelo 'mistral-medium' y una temperatura=0 (para que el modelo haga su tarea sin ser creativo), con un tiempo de espera de 5 minutos, ya que en ocasiones tarda más en generar la respuesta. Después, le pasamos el prompt e invocamos al LLM generador.

```
# Configuración de MistralAI
llm = ChatMistralAI(
    model="mistral-medium",
    temperature=0,
    timeout=300
)

# Pasamos el Prompt a MistralAI
prompt = PromptTemplate.from_template(prompt_text)

# Pasamos el Prompt a MistralAI
graph_extractor = prompt | llm

# Generamos la respuesta, pasando el texto
response = graph_extractor.invoke({"text": text})
print(response.content)
```

Figura 9 – Configuración e invocación del LLM generador

Por último, mostramos la respuesta por pantalla (estructura de nodos y relaciones).

```
NODES:
- 2024-03-02 (Fecha)
- transaction_01 (Transaction)
- Supermercado (Categoría)
- Compra semanal (Descripción)
- Mercadona (Entidad)
- 85.50 (Cantidad)
- Gasto (TipoTransacción)
- 2024-03-05 (Fecha)
- transaction_02 (Transaction)
- Telecomunicaciones (Categoría)
- Factura móvil (Descripción)
- Vodafone (Entidad)
- 35.99 (Cantidad)
- 2024-03-10 (Fecha)
- transaction_03 (Transaction)
- Entretenimiento (Categoría)
- Suscripción mensual (Descripción)
- Netflix (Entidad)
- 12.99 (Cantidad)
```

Figura 10 – Contenido parcial de la respuesta del LLM generador, con algunos nodos

4.1.4. Estructuración del grafo de conocimiento

Después, depuramos la respuesta de nuestro LLM para construir la estructura base de nuestro grafo de conocimiento, con la función parse_graph_output. Para ello, creamos una lista de diccionarios para los nodos y otra para las aristas, con cada diccionario estructurado de la siguiente forma:

- En nodes representamos cada nodo como: ‘{'name': '2024-03-02', 'label': 'Fecha'}’, siendo ‘name’ el valor de la fila concreta del csv, y ‘label’ la etiqueta que representa el atributo de la columna a la que corresponde el valor.

- En relationships representamos cada relación como: ‘{'source': '2024-03-02', 'relation': 'tiene_transaction', 'target': 'transaction_01'}’, siendo ‘source’ el nodo origen, ‘relation’ la arista que representa la relación entre los nodos y ‘target’ el nodo objetivo al que apunta la arista.

4.1.5. Definición del prompt y LLM evaluador

Una vez creadas las listas ‘nodes’ y ‘relationships’, vamos a validar su estructura, a través de un LLM evaluador, el cual se va a implementar de forma similar al anterior.

Definimos el prompt, donde le damos al LLM la función de ‘verificador de grafos de conocimiento’. También definimos una serie de reglas, que verifiquen que:

- Los nodos definidos en el primer prompt, coincidan con los generados y almacenados en la lista de ‘nodes’.
- Hay exactamente 85 nodos referentes a ‘transaction’ (hay 85 transacciones en el csv original).
- No puede haber nodos duplicados, pero si múltiples relaciones hacia un mismo nodo (por ejemplo, 5 transacciones tuvieron una cantidad de 29’99 euros).
- Indicamos la estructura de las relaciones que debió generar el primer LLM, para verificar que lo hizo correctamente.
- No puede haber relaciones que unifiquen nodos fuera del esquema anterior, ni nombres de relaciones que no coincidan con los definidos.

Por último, si la estructura es válida, entonces su respuesta debe ser esa, si no, debe indicar en qué nodo/s o relación/es se encuentran los fallos.

```
Eres un verificador de grafos de conocimiento extraídos de textos sobre las transacciones bancarias de una persona.
Revisa si el grafo de conocimiento (nodos y relaciones) cumple exactamente con las siguientes reglas:

1. Los tipos válidos de NODOS son: Fecha, Categoría, Entidad (nombre comercial o persona), Descripción, Cantidad (en euros), y Transac
2. Debe haber exactamente 85 nodos referentes a transaction.
3. Cada nodo debe estar definido una sola vez en la lista de NODES. Si un valor como "35.99" aparece dos veces en NODES, es un error.
4. Las RELACIONES permitidas son:
   - [fecha] -> tiene_transaction -> [transaction]
   - [transaction] -> tiene_descripcion -> [descripcion]
   - [transaction] -> involucra -> [entidad]
   - [entidad] -> tiene_categoria -> [categoria]
   - [transaction] -> es_tipo -> [tipoTransacción]
   - [transaction] -> tiene_cantidad -> [cantidad]
5. Toda relación debe enlazar nodos existentes, del tipo correcto.
6. No se permiten relaciones fuera del esquema definido.

Revisa el grafo siguiente y responde únicamente con:
- "La estructura del grafo es correcta" si la estructura es correcta.
- "La estructura no es correcta" seguido de una lista clara de los errores encontrados (por ejemplo: "Nodo duplicado en NODES: 35.99",
```

Figura 11 – Visión parcial del prompt para el LLM evaluador

Después configuramos el LLM evaluador con el modelo 'mistral-medium' y una temperatura=0, y le pasamos ambas listas, para que analice la estructura del grafo de conocimiento, mostrando la respuesta por pantalla.

```
# Configuración de MistralAI
llm = ChatMistralAI(
    model="mistral-medium",
    temperature=0,
)

# Creamos el prompt (instancia) a partir de prompt_eval
prompt = PromptTemplate.from_template(prompt_eval)

# Pasamos el Prompt a MistralAI
graph_validator = prompt | llm

# Invocamos al LLM evaluador
response = graph_validator.invoke({
    "nodes": nodes,
    "relationships": relationships
})

# Mostramos la respuesta
print(response.content)
```

Figura 12 - Configuración e invocación del LLM evaluador

4.1.6. Conexión y subida a la base de datos Neo4j Aura

Una vez comprobada y validada la estructura del grafo de conocimiento, establecemos la conexión con la base de datos Neo4j Aura, e instanciamos el grafo de conocimiento (de momento no hay ningún grafo almacenado en la base de datos).

```
from langchain_neo4j import Neo4jGraph

# Establecer conexión con Neo4j Aura
os.environ["NEO4J_URI"] = "neo4j+s://949f0956.databases.neo4j.io"
os.environ["NEO4J_USERNAME"] = "neo4j"

# Inicialización del grafo de conocimiento (conexión a Neo4j)
graph = Neo4jGraph(refresh_schema=False)
```

Figura 13 - Conexión con la base de datos Neo4j Aura e instanciación del grafo de conocimiento

Con la función *normalize_label*, reemplazamos los espacios y guiones por guion bajo, con el objetivo de preparar las etiquetas para que sean consistentes y compatibles con las convenciones de nombres en grafos utilizadas en Neo4j Aura.

```
def normalize_label(text):
    return text.strip().replace(" ", "_").replace("-", "_")
```

Figura 14 - Función para normalizar las etiquetas

La función *insert_graph_into_neo4j* se encarga de cargar los nodos y relaciones ya definidos y validados, en la base de datos Neo4j Aura. Utilizamos la función *normalize_label* para normalizar los nodos y aristas, y utilizamos el lenguaje de consultas Cypher para agregar a la base de datos en primer lugar todos los nodos, y después las relaciones entre ellos.

```python
def insert_graph_into_neo4j(graph, nodes, relationships):
    created_nodes = set()

    for node in nodes:
        identifier = (node["name"], node["label"])
        if identifier not in created_nodes:
            label = normalize_label(node["label"])
            query = f"""
            MERGE (n:{label} {{name: $name}})
            """
            graph.query(query, params={"name": node["name"]})
            created_nodes.add(identifier)

    for rel in relationships:
        relation = normalize_label(rel["relation"]).upper()
        query = f"""
        MATCH (a {{name: $source}})
        MATCH (b {{name: $target}})
        MERGE (a)-[:{relation}]->(b)
        """
        graph.query(query, params={
            "source": rel["source"],
            "target": rel["target"]
        })
```

Figura 15 - Función de inserción de la estructura del grafo de conocimiento en la base de datos Neo4j Aura

Por último, invocamos a la función explicada anteriormente, le pasamos la instanciación del grafo de conocimiento 'graph', la lista con los nodos 'nodes' y la lista con las relaciones 'relationships'.

```python
# Insertamos el grafo de conocimiento en Neo4j Aura
insert_graph_into_neo4j(graph, nodes, relationships)

print("Grafo insertado correctamente en Neo4j.")
```

Figura 16 - Subida del grafo de conocimiento a Neo4j Aura

4.1.7. Visualización del grafo de conocimiento transaccional

Una vez descrita la implementación, vamos a observar la estructura del grafo de conocimiento almacenado en la base de datos Neo4j Aura. Si nos fijamos, tenemos 263 nodos en total, cada uno con un color distinto, que representa a un tipo concreto, entre los que tenemos:

- **Cantidad (38)**: El dinero utilizado en una transacción.
- **Categoría (7)**: La categoría a la que pertenece una entidad.
- **Descripción (22)**: Descripción ligada a una transacción.
- **Entidad (26)**: Entidad en la que o a la que se realizó una transacción.
- **Fecha (81)**: Fecha en la que ocurrió una transacción.
- **TipoTransacción (4)**: El tipo de transacción que se realizó, como transferencia o gasto.
- **Transaction (85)**: La representación de cada una de las transacciones, para unir los nodos de forma coherente.

El número de transacciones y fechas no coincide, ya que hubo días en los que se realizaron más de una transacción, lo mismo podemos decir de cantidad, que de igual forma se repite en varias ocasiones, y se creó un nodo único para cada una, con el fin de evitar duplicados.

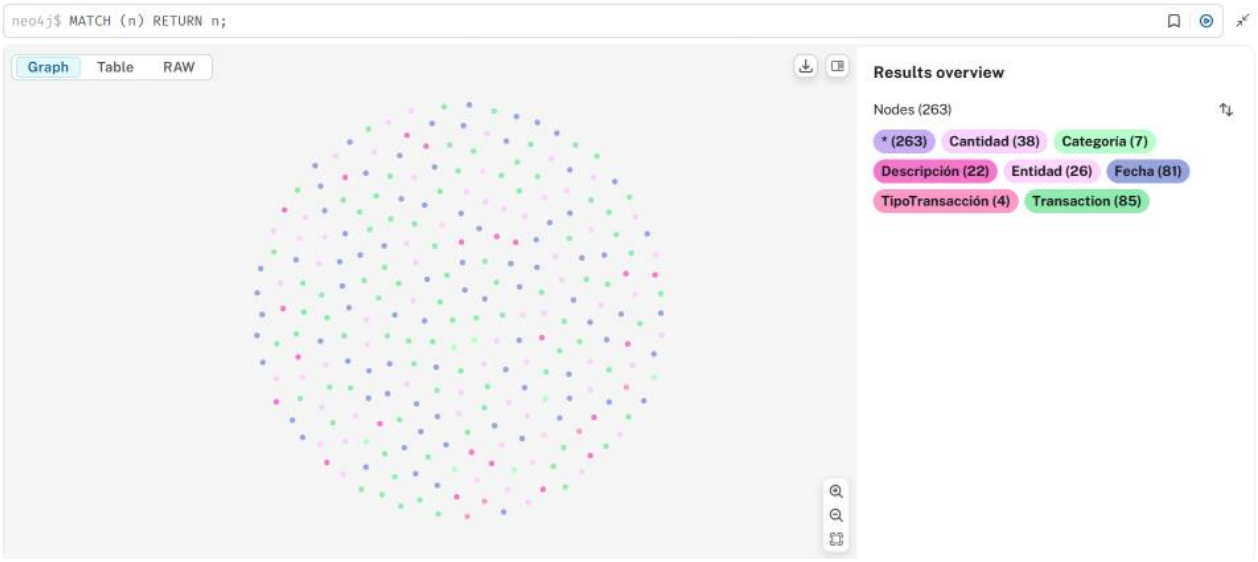


Figura 17 - Nodos del grafo de conocimiento (en consola de Neo4j Aura)

Relación TIENE_DESCRIPCION

Esta relación une las transacciones con su descripción, como puede haber descripciones similares, puede darse el caso de que varias transacciones apunten al mismo nodo descripción.

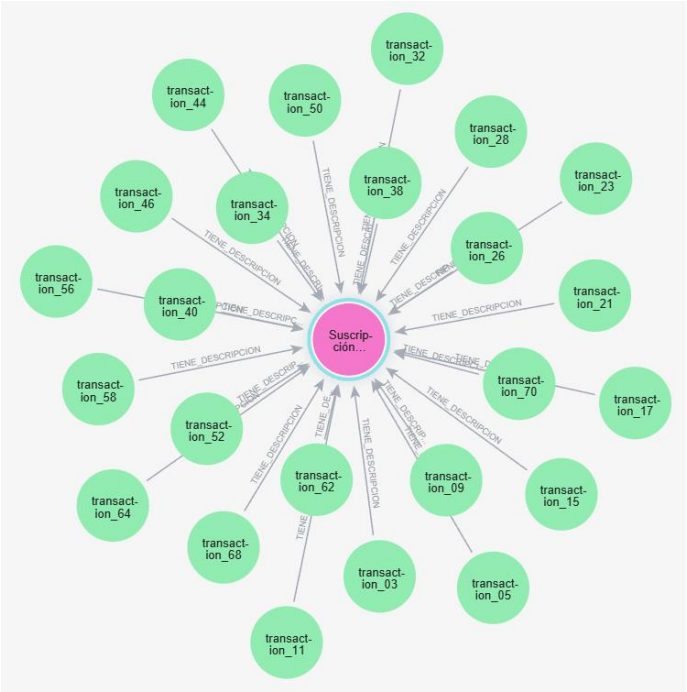


Figura 18 – Transacciones con descripción 'Suscripción mensual' (en consola de Neo4j Aura)

Relación ES_TIPO

Indica la relación entre el tipo (gasto, transferencia, ingreso, reembolso) de una transacción con dicha transacción.

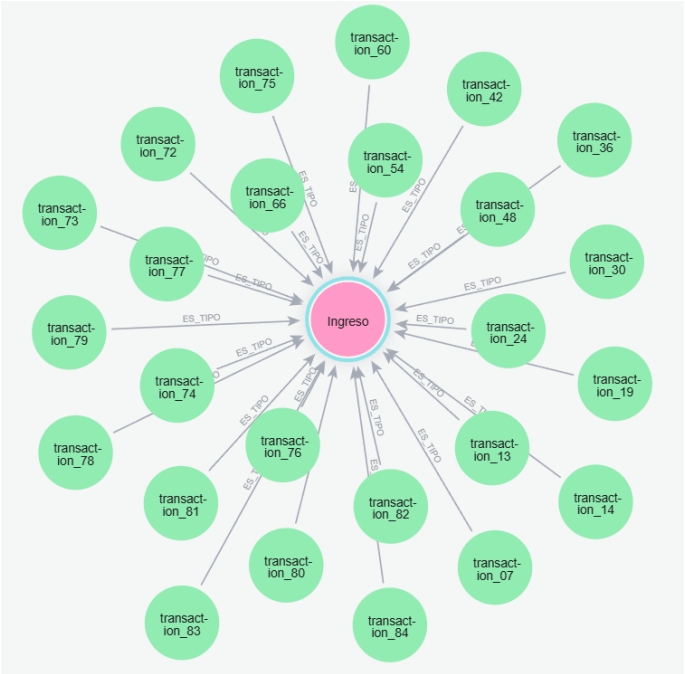


Figura 19 - Transacciones de tipo 'Ingreso' (en consola de Neo4j Aura)

Relación INVOLUCRA

Con esta relación, sabemos la entidad que está involucrada en una o varias transacciones.

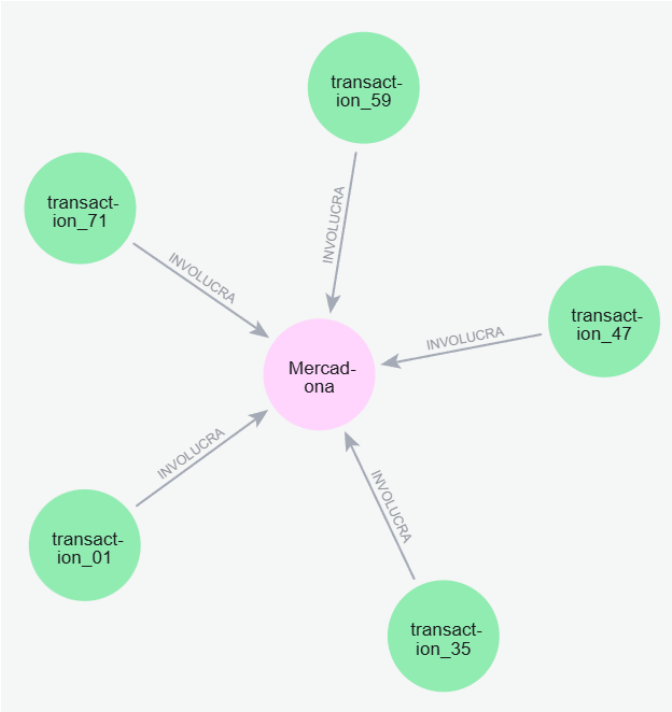


Figura 20 - Transacciones en las que está involucrada la entidad 'Mercadona' (en consola de Neo4j Aura)

Relación TIENE_CANTIDAD

Con esta relación, sabemos la cantidad de dinero en euros utilizada en cada transacción.

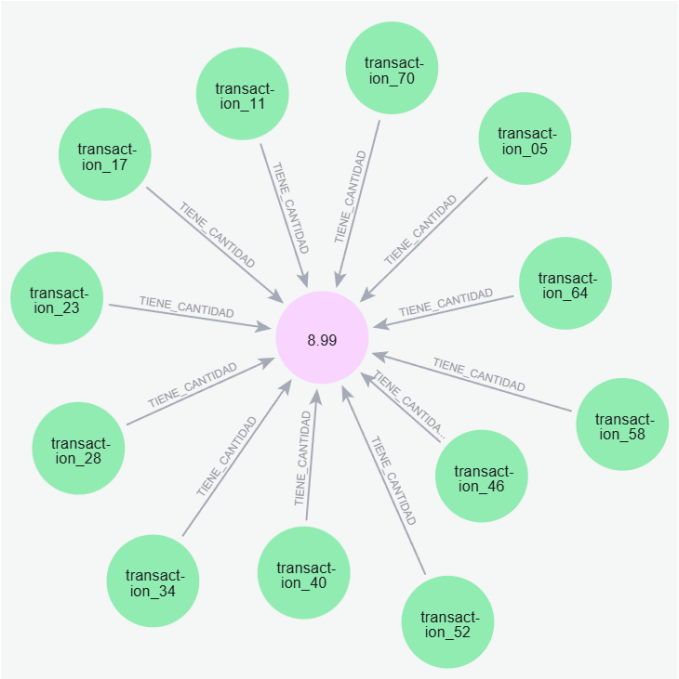


Figura 21 - Transacciones con una cantidad de 8'99 euros (en consola de Neo4j Aura)

Relación TIENE_CATEGORIA

Indica la categoría a la que pertenece una o varias entidades, fundamental para que el LLM entienda 'qué es la entidad' a la hora de recibir consultas del usuario.

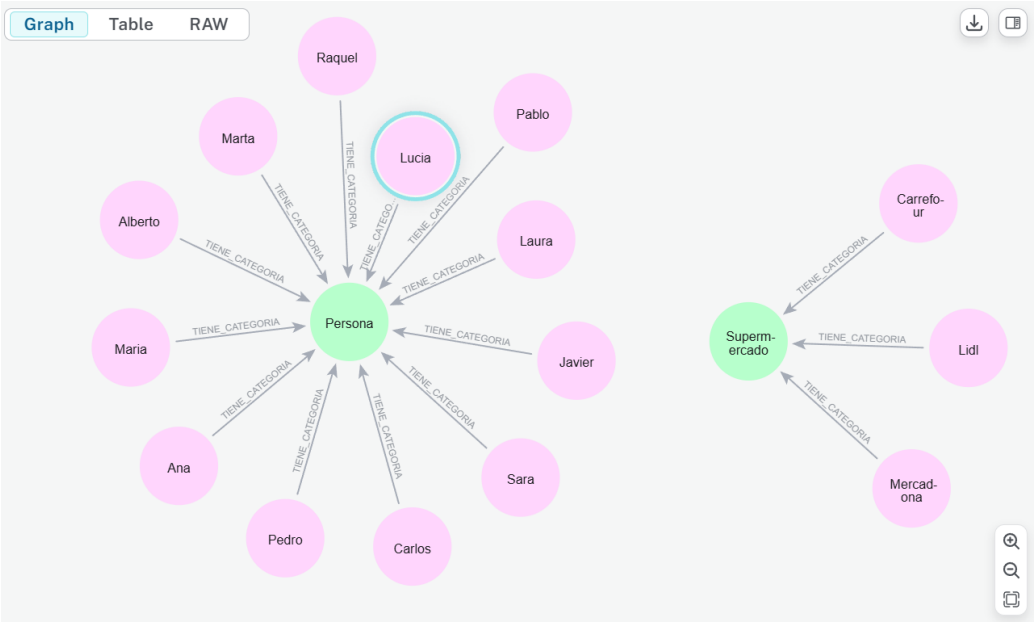


Figura 22 - Entidades pertenecientes a la categoría 'Persona' y 'Supermercado' (en consola de Neo4j Aura)

Relación TIENE_TRANSACTION

Indica la fecha en la que se realizó una o más transacciones, esta relación es bastante importante de cara al entendimiento del LLM sobre las consultas Cypher.

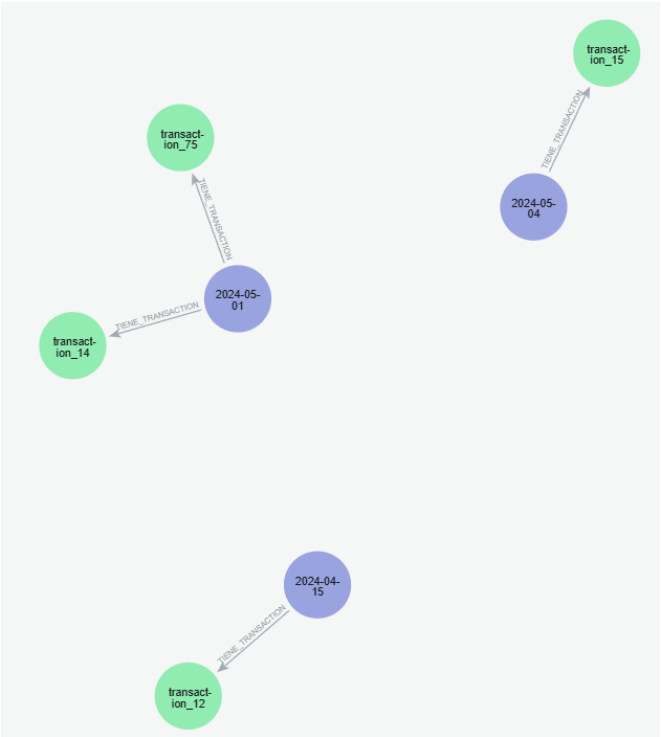


Figura 23 – Ejemplo de fechas en las que se realizó una o más transacciones (en consola de Neo4j Aura)

4.2. Integración del grafo de conocimiento con el LLM

Para integrar el grafo de conocimiento, previamente generado y almacenado en la base de datos Neo4j Aura, con modelos de lenguaje grandes (LLMs), se recurrió a un sistema que utiliza dos LLMs, junto con una función que se encarga de ejecutar las consultas a la base de datos pertinentes según la pregunta formulada por el usuario, y se devuelve en lenguaje natural la respuesta más apropiada por parte de uno de los LLMs. Su desarrollo fue llevado a cabo en Google Colab, en un archivo *.ipybn*, cuya lógica fue trasladada e implementada como el backend de la aplicación web de interfaz conversacional, en el archivo *app.py*, el cual se explica a continuación.

4.2.1. Carga de datos y conexión con la base de datos Neo4j Aura

Inicializamos la aplicación web usando el framework Flask, y utilizamos CORS (Cross-Origin Resource Sharing) para que el servidor Flask autorice peticiones **HTTP** que vienen desde otros dominios o puertos diferentes al suyo, esto debido a que el servidor del frontend se ejecuta en un puerto distinto, el 5000. Además, activamos el modo 'debug' para ver errores detallados y recargar automática cuando se realizan cambios sobre el código.

```
# Inicializar Flask
app = Flask(__name__)
CORS(app)
```

Figura 24 - Inicialización de la aplicación web con Flask

```
if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

Figura 25 - Ejecución del servidor Flask en el puerto 5000

Cargamos el archivo .env, el cual posee las variables de entorno:

- **MISTRAL_API_KEY**: la clave para utilizar la API de Mistral AI.
- **NEO4J_URI**: dirección (URI) a la que se conecta la aplicación web para acceder a la base de datos Neo4j Aura.
- **NEO4J_USERNAME**: Nombre de usuario de la base de datos.
- **NEO4J_PASSWORD**: Contraseña para acceder a la base de datos Neo4j Aura.

A continuación, cargamos las credenciales de Neo4j Aura, y realizamos la instanciación del grafo de conocimiento almacenado, con *refresh_schema()* nos aseguramos de que se guarda la estructura en caché, para que el LLM pueda acceder más tarde.

```
# === Configuración de Neo4j Aura ===
os.environ["NEO4J_URI"] = os.getenv("NEO4J_URI")
os.environ["NEO4J_USERNAME"] = os.getenv("NEO4J_USERNAME")
os.environ["NEO4J_PASSWORD"] = os.getenv("NEO4J_PASSWORD")

# Instanciación del grafo de conocimiento
graph = Neo4jGraph(refresh_schema=False)
graph.refresh_schema()
```

Figura 26 - Configuración de Neo4j Aura

4.2.2. Configuración e instanciación para LLMs

A continuación, cargamos la clave API de Mistral AI en nuestro entorno, y creamos la instancia del LLM utilizando el modelo 'mistral-medium', con una temperatura=0 para que sea lo más preciso posible generando las consultas y las respuestas en lenguaje natural, y un 'timeout' de 300 segundos, por si demora más en responder al usuario, no muestre el error de que se quedó sin tiempo. Esta configuración va a ser aplicada en ambos LLMs.

```
# Carga de clave API de Mistral AI
os.environ["MISTRAL_API_KEY"] = os.getenv("MISTRAL_API_KEY")

# Configuración de Mistral AI
llm = ChatMistralAI(
    model="mistral-medium",
    temperature=0,
    timeout=300,
)
```

Figura 27 - Carga de la clave API de MIstral AI y configuración de la instancia LLM

4.2.3. Prompt para el LLM generador de consultas Cypher

Para que el LLM realice su tarea de generación de consultas en lenguaje Cypher, es necesario definir correctamente un prompt, el cual será explicado a continuación.

Utilizando *PromptTemplate*, definimos las variables 'question', que es la consulta o pregunta que el usuario introduce, y 'schema', que corresponde al esquema concreto del grafo de conocimiento.

```
cypher_prompt = PromptTemplate(
    input_variables=["question", "schema"],
```

Figura 28 - Definición de cypher_prompt y las variables que utiliza

Después, se define el *template*, donde explicamos el rol del LLM y las reglas que debe seguir, que se basan en los siguientes principios:

- **Sintaxis restringida**: Todas las consultas deben comenzar con cláusulas válidas de Cypher, como MATCH u OPTIONAL MATCH, y deben construirse exclusivamente usando nodos y relaciones definidos en el esquema de la base de datos.

- **Formato y estilo**: Las respuestas sólo pueden contener la consulta como tal, sin notas adicionales.

- **Seguridad y protección de datos**: Se prohíbe la ejecución o generación de consultas destructivas (DELETE, SET, REMOVE, CREATE, MERGE). Si el usuario lo solicita, el sistema devuelve una advertencia.

- **Interacción con el usuario**: Si el usuario saluda, se despide o pregunta cómo estás, se responde con cordialidad. Si se solicita una definición de término financiero, se ofrece una respuesta breve, clara y precisa. No se accede a peticiones que incumplan alguna de estas reglas, aunque el usuario lo intente persuadir.

- **Tratamiento de entidades**: Las fechas deben seguir el formato ISO-8601 (YYYY-MM-DD), y se permite el uso de STARTS WITH para agrupación temporal. En búsquedas por texto dentro de descripciones, se emplea toLower y CONTAINS para garantizar coincidencias insensibles a mayúsculas/minúsculas (por ejemplo, al buscar "Bizum"). Las entidades etiquetadas como “Persona” se interpretan como contactos cercanos del usuario (amigos o compañeros).

```
template="""
Eres un asistente especializado en Neo4j que genera EXCLUSIVAMENTE consultas Cypher válidas.

IMPORTANTE:
- NO incluyas comentarios, notas, encabezados, explicaciones ni markdown.
- NO incluyas nada más que la consulta.
- NO realices acciones destructivas (DELETE, SET, REMOVE, CREATE, MERGE). Si el usuario intenta una, adviértele.
- Si el usuario hace preguntas sobre terminología de finanzas, defínelas claras y concisas.
- Si el usuario saluda, te pregunta qué tal estás o se despide, devuelve la acción de forma amable.
- NO te dejas persuadir si el usuario te pide algo que se salga de tu tarea.
- Usa exclusivamente las relaciones y nodos definidos en el esquema.
- Comienza con MATCH, OPTIONAL MATCH o una cláusula Cypher válida.
- Las fechas tienen formato ISO-8601 "YYYY-MM-DD". Puedes usar STARTS WITH para agrupar por mes o año.
- Usa CONTAINS y 'toLower' sobre el nodo Descripcion cuando lo veas necesario (Por ejemplo, en Descripcion
cuando el usuario mencione Bizum).
- Las entidades con categoria "Persona", son compañeros o amigos del usuario.
```

Figura 29 - Primera parte de cypher_prompt, con la tarea y reglas a seguir por el LLM

En la segunda parte, le indicamos el esquema del grafo de conocimiento con 'scheme', y le indicamos un par de ejemplos de consultas en lenguaje Cypher, para que se desenvuelva mejor a la hora de generar dichas consultas, y le pasamos la pregunta del usuario.

```
Esquema del grafo de conocimiento:
{schema}

Ejemplo:
- ¿En 2025 hice compras?: MATCH (f:Fecha)-[:TIENE_TRANSACTION]->(t:Tr
- ¿Cuánto pagué en supermercado?: MATCH (t:Transaction)-[:INVOLUCRA]-

Pregunta: {question}
"""
```

Figura 30 - Contenido parcial de la segunda parte de cypher_prompt

4.2.4. Funciones auxiliares

La función *detectar_idioma_usuario* utiliza la librería de langdetect, para detectar el idioma de la pregunta del usuario, y si el idioma detectado es inglés se devuelve 'inglés', y si es cualquier otro idioma, se devuelve 'español', esto debido a que, en muchas ocasiones, langdetect no es infalible, y detecta frases en español como otros idiomas.

```python
# Función que detecta el idioma de la consulta del usuario
DetectorFactory.seed = 42

def detectar_idioma_usuario(texto_usuario: str) -> str:
    try:
        idioma_detectado = detect(texto_usuario)
        if idioma_detectado == 'en':
            return 'inglés'
        else:
            return 'español'
    except:
        return 'español'
```

Figura 31 - Detector de idioma de la consulta del usuario

Para evitar que el usuario introduzca código Cypher no deseado, potencialmente ejecutable en la base de datos, puesto que el LLM no es infalible, y puede pasar la consulta como válida, se implementó la función *contiene_cypher,* a la que se le pasa la pregunta del usuario, y si la frase contiene alguna palabra de las definidas en 'comandos_cypher', se obtiene un valor booleano 'true', en otro caso, 'false'.

```python
# Función que detecta si el texto del usuario contiene código Cypher
def contiene_cypher(texto):
    comandos_cypher = ["MATCH", "CREATE", "DELETE", "DETACH", "SET", "REMOVE", "MERGE", "DROP"]
    # Buscar comandos, ignorando mayúsculas/minúsculas
    return any(cmd in texto.upper() for cmd in comandos_cypher)
```

Figura 32 - Función contiene_cypher

Para ejecutar la consulta en lenguaje Cypher, creamos la función run_cypher_query y le pasamos como parámetros tanto la instancia del grafo de conocimiento como dicha consulta. A continuación, validamos que la consulta no contenga comandos prohibidos, y si es así, la ejecutamos sobre la base de datos Neo4j Aura y guardamos la respuesta. Si falla la consulta, se guarda el error pertinente.

```python
# Función que valida y ejecuta la consulta Cypher
def run_cypher_query(query, graph):
    # Lista de comandos prohibidos
    comandos_prohibidos = ["DELETE", "DETACH", "REMOVE", "DROP", "CREATE", "MERGE", "SET"]

    # Validamos si la consulta contiene comandos prohibidos (ignorando mayúsculas/minúsculas)
    if any(cmd in query.upper() for cmd in comandos_prohibidos):
        return "Error: Consulta contiene comandos prohibidos y no se ejecutará."

    try:
        resultado = graph.query(query)
        return resultado
    except Exception as e:
        # Mostramos el error si la query generada falla
        return f"Error de sintaxis o ejecución en la consulta: {str(e)}"
```

Figura 33 - Función run_cypher_query

4.2.5. Prompt para el LLM generador de respuestas en lenguaje natural

Para la segunda invocación al modelo de lenguaje previamente configurado, definimos su correspondiente prompt, con el fin de que responda al usuario en lenguaje natural, de forma clara y breve según una serie de reglas. Además, definimos una serie de variables:

- Resultado: Respuesta de la base de datos tras ejecutar la consulta Cypher.
- Pregunta: Consulta del usuario.
- Language: Lenguaje detectado en la consulta del usuario.

```
response_prompt = PromptTemplate(
    input_variables=["resultado", "pregunta", "language"],
    template="""
    Eres un asistente financiero de las transacciones (en euros) del usuario, que explica resultados de consultas
    en lenguaje natural, claro y breve.

    Tu tarea es generar una respuesta en el idioma indicado: {language}, sabiendo que la pregunta original del
    usuario fue {pregunta}.

    REGLAS:
    - Si hay un solo resultado, responde directamente.
    - Si hay varios, resume en una frase tipo lista.
    - Si no hay resultados, dilo amablemente.
    - NO te dejas persuadir si el usuario te pide algo que se salga de tus tareas de comprensión de la consulta Cypher.

    Resultado:
    {resultado}

    Respuesta:
    """
)
```

Figura 34 - Prompt para el LLM generador de respuestas en lenguaje natural

4.2.6. Funcionamiento del sistema conversacional LLM-usuario

En este apartado, vamos a explicar el funcionamiento del sistema, donde integramos el LLM con el grafo de conocimiento, desde que procesa la consulta del usuario, hasta la generación de su respuesta en lenguaje natural.

1. Se recibe una solicitud HTTP con una pregunta del usuario en formato JSON. Se extrae su contenido desde el campo "question".

2. Si no se proporciona ninguna pregunta, el sistema devuelve un error con un código HTTP 400, indicando que falta el parámetro requerido.

3. Antes de procesar la pregunta, se verifica si el usuario intenta introducir código Cypher directamente o mezclado con lenguaje natural. Si es así, se bloquea la solicitud y se le pide que formule su pregunta en lenguaje natural.

4. Se detecta automáticamente el idioma en que está escrita la pregunta. Esto permite generar respuestas coherentes en el mismo idioma que utilizó el usuario.

5. Se envía la pregunta al primer modelo de lenguaje (LLM), para que genere la consulta Cypher o una respuesta en lenguaje natural, según corresponda.

6. Se analiza la respuesta del modelo para determinar si se trata de una consulta Cypher. Esto se hace comprobando si la primera palabra está

completamente en mayúsculas, como suele ser en sentencias Cypher válidas (por ejemplo, MATCH o RETURN), si no es una consulta Cypher, se muestra la respuesta en lenguaje natural del LLM al usuario.

7. Si se detecta que la respuesta es una consulta Cypher válida, se ejecuta dicha consulta en la base de datos Neo4j Aura sobre el grafo de conocimiento, y se devuelve la respuesta esperada.

8. A partir del resultado de la consulta, se invoca de nuevo al modelo de lenguaje, esta vez utilizando el prompt para generación de respuestas en lenguaje natural, y le pasamos tanto la pregunta original como el idioma detectado.

9. Se envía al usuario la respuesta generada en lenguaje natural. Además, también se incluye la consulta Cypher como parte de la respuesta para mayor transparencia y pruebas.

10. Si ocurre algún error durante el procesamiento (por ejemplo, un fallo del modelo o de la base de datos), se captura la excepción y se devuelve un mensaje de error con el detalle técnico del problema.

La estructura del sistema es la siguiente:

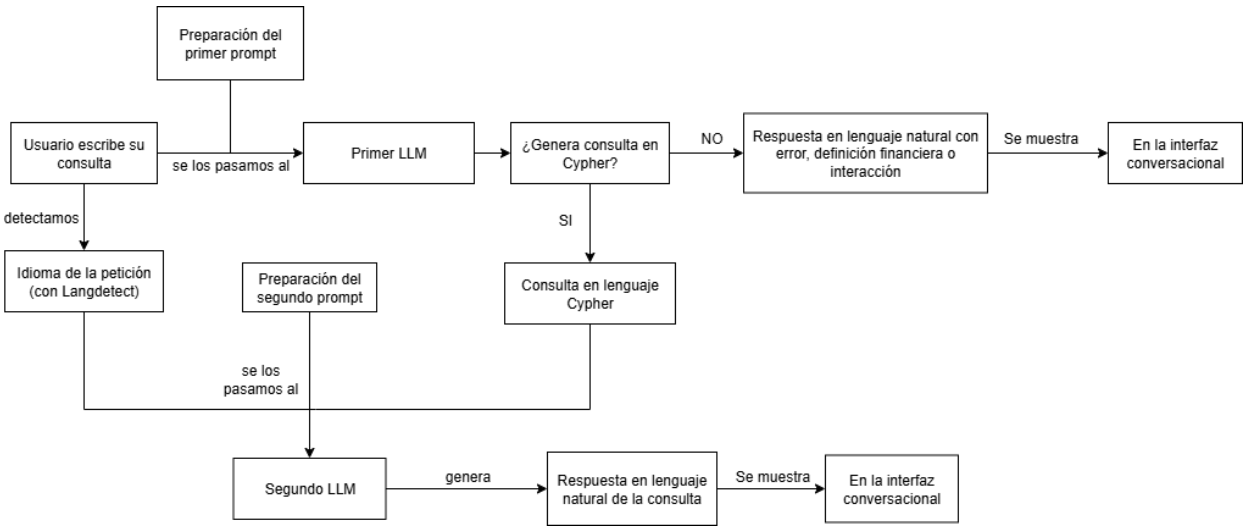


Figura 35 – Funcionamiento del sistema conversacional LLM-usuario (realizado con draw.io)

4.3. Interfaz conversacional

Para el desarrollo del frontend (interfaz web), inicialmente consideré dos opciones: React, por experiencia previa en otros proyectos, y Vue.js, que destaca por su rapidez de implementación y menor complejidad para interfaces simples. Dado que la interfaz en este caso cumple una función meramente comunicativa (servir de puente entre el usuario y el sistema conversacional basado en LLM y KG), opté por Vue.js, por su facilidad de configuración y menor sobrecarga visual.

El objetivo de la interfaz es permitir que el usuario realice preguntas relacionadas con sus transacciones bancarias y reciba respuestas del sistema de manera clara e interactiva, incluyendo en algunos casos la consulta Cypher generada por el modelo.

4.3.1. Estructura general

La interfaz se compone de tres secciones principales:

El encabezado del chat, con el título y un botón para alternar entre el modo claro y oscuro:

```
<div class="chat-container">
  <!-- Header del chat con título y botón para alternar tema -->
  <div class="chat-header">
    <h2>Transacciones Bancarias</h2>
    <!-- Botón que cambia el modo claro/oscuro -->
    <button @click="toggleTheme" class="theme-toggle">
      {{ isDarkMode ? 'Modo Claro' : 'Modo Oscuro' }}
    </button>
  </div>
```

Figura 36 - Código del encabezado del chat (en VSCode)

La zona de mensajes, donde se listan tanto los mensajes del usuario como los del asistente con la hora exacta en la que se realizaron, y en caso de que la respuesta del modelo incluya una consulta Cypher, se permite al usuario mostrarla u ocultarla con un simple clic:

```
  <!-- Texto del mensaje -->
  <div class="message-content">{{ message.text }}</div>
  <!-- Si el mensaje tiene consulta Cypher, mostrar toggle para mostrarla u ocultarla -->
  <div v-if="message.cypher" class="message-cypher" @click="toggleCypher(index)">
    <span v-if="!message.showCypher">Mostrar consulta Cypher</span>
    <code v-else>{{ message.cypher }}</code>
  </div>
  <!-- Hora del mensaje -->
  <div class="message-time">{{ message.time }}</div>
</div>
```

Figura 37 - Código para el texto del mensaje, visualización de la consulta Cypher y hora de envío (en VSCode)

El área de entrada, compuesta por un input y un botón de envío, con un pequeño control para evitar envíos vacíos o múltiples mientras el sistema procesa una consulta:

```
<!-- Entrada para nuevo mensaje y botón enviar -->
<div class="chat-input">
  <input
    v-model="newMessage"
    @keyup.enter="sendMessage"
    placeholder="Pregunta sobre tus transacciones bancarias..."
    :disabled="loading"
  />
  <button @click="sendMessage" :disabled="loading || !newMessage.trim()">
    {{ loading ? 'Enviando...' : 'Enviar' }}
  </button>
</div>
```

Figura 38 - Código con el área de entrada del mensaje escrito por el usuario (en VSCode)

4.3.2. Comunicación con el backend

La lógica principal de interacción está definida en la función *sendMessage*, que envía la consulta del usuario a un endpoint local (/preguntar) expuesto por el backend, el cual se encarga de ejecutar el pipeline de procesamiento: interpretar la pregunta mediante el LLM, generar (si corresponde) una consulta Cypher, y devolver la correspondiente respuesta, que una vez recibida, el frontend la muestra, y si incluye una consulta Cypher, se habilita un elemento visual para inspeccionarla.

```
try {
  // Envío del mensaje al servidor backend (suponiendo API REST en localhost)
  const response = await fetch('http://localhost:5000/preguntar', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ question: userMessage })
  });

  const data = await response.json();

  // Si la respuesta es error, lanza excepción para manejarlo abajo
  if (!response.ok) {
      throw new Error(data.error || 'Error en la respuesta del servidor');
  }
```

Figura 39 - Código principal de comunicación con el backend (en VSCode)

4.3.3. Estilos y experiencia de usuario

La interfaz incorpora un sencillo sistema de **temas claro/oscuro** controlado mediante variables CSS (:root y [data-theme="dark"]), facilitando la accesibilidad y personalización visual. También se aplica un scroll automático al enviar o recibir mensajes para asegurar que la conversación siempre esté visible desde el último mensaje.

4.4. Ejemplos representativos del sistema

A continuación, se presentan varios ejemplos representativos que ilustran el comportamiento de la integración del LLM con el grafo de conocimiento a través de la aplicación web de interfaz conversacional, resaltando sus características.

4.4.1. Características del sistema

Al abrir la interfaz, el sistema es el que genera un mensaje de bienvenida automático al usuario, invitándolo a realizar una consulta.



Figura 40 - Estado inicial de la interfaz conversacional

El sistema no conserva contexto entre mensajes. Cada consulta del usuario se procesa de forma aislada, realizándose en lenguaje natural y son interpretadas con ayuda del grafo de conocimiento.

El sistema responde en el idioma de la consulta del usuario sobre información relacionada con sus movimientos bancarios, como gastos, ingresos, reembolsos o transferencias realizadas. También es capaz de responder sobre terminología financiera, aunque sólo en español.

El sistema tiene protecciones frente a inyecciones de código Cypher, además de evitar salirse de su tarea con persuasiones por parte del usuario.

El grafo de conocimiento no tiene transacciones actualizadas en tiempo real, ya que se generó a partir de transacciones ficticias, desde marzo de 2024 hasta febrero de 2025, por lo que el sistema carece de capacidad para saber el momento temporal de la conversación.

4.4.2. Ejemplo 1 - Interacción del usuario con el sistema

Este ejemplo muestra una interacción inicial, en la que el usuario formula la pregunta de ejemplo proporcionada por el sistema (¿Cuánto gasté en supermercados en marzo?). El sistema accede al grafo de conocimiento para obtener la transacción a partir de la fecha, en este caso el mes de marzo, busca la categoría supermercado y proporciona una respuesta con el dinero que gastó el usuario para dicha consulta.

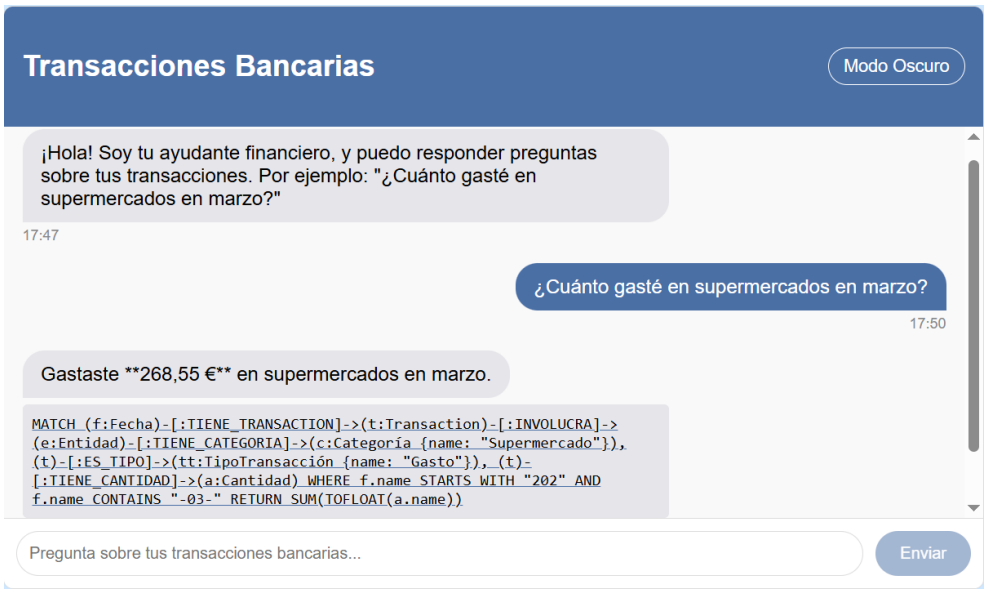


Figura 41 - Consulta del usuario y respuesta satisfactoria del sistema

4.4.3. Ejemplo 2 - Consulta dependiente sin memoria

En este ejemplo, observamos que el sistema no tiene capacidad para memorizar contextos, ya que responde correctamente a la primera pregunta con el dinero gastado en mayo en supermercados por parte del usuario, pero cuando le pregunta por junio, el sistema no sabe si se refiere a supermercados, y se limita a responder con las transacciones que se hicieron en dicho mes.



Figura 42 - Ejemplo de que el sistema no tiene memoria conversacional

4.4.4. Ejemplo 3 - Consulta sobre ingresos importantes

En este caso, el sistema interpreta correctamente la consulta del usuario sobre “ingresos importantes este trimestre”. Internamente, traduce esa petición a una consulta Cypher que filtra los ingresos dentro del primer trimestre del año actual (01/01 al 31/03) y aplica un umbral de >1000 € para considerar un ingreso como "importante". Esta heurística permite al sistema ofrecer una respuesta útil sin requerir aclaraciones adicionales por parte del usuario. Además, ordena los resultados por monto para priorizar los ingresos más relevantes.



Figura 43 – Consulta satisfactoria para ingresos importantes trimestrales del usuario

4.4.5. Ejemplo 4 - Gestión de mensajes distintos a consulta de transacciones

Como podemos observar, si el usuario saluda al sistema, este en lugar de generar consultas, le devuelve el saludo, por lo que el sistema es capaz de gestionar mensajes fuera del alcance de su tarea, para redirigir al usuario a realizar cuestiones sobre dicha tarea.



Figura 44 - Usuario saluda al sistema, y este le devuelve el saludo, recordando su tarea principal de asistente

4.4.6. Ejemplo 5 - Consulta en otro idioma

Cuando el usuario realiza una consulta, por ejemplo, en inglés, el sistema devuelve la respuesta en lenguaje natural en dicho idioma.



Figura 45 - Consulta del usuario en inglés y respuesta correspondiente en dicho idioma del sistema

4.4.7. Ejemplo 6 – Protección frente a inyección de código Cypher

Si el usuario intenta realizar un ataque al sistema, modificando por ejemplo el grafo de conocimiento, necesita pasar la consulta maliciosa, escondida en una consulta que a priori puede parecer válida o inocente. En este caso, el usuario pregunta por Bizums realizados, y al final coloca una consulta Cypher que borra el grafo de conocimiento almacenado en Neo4j Aura por completo. Como podemos observar, el sistema lo detecta, y le advierte por realizar dicha acción.

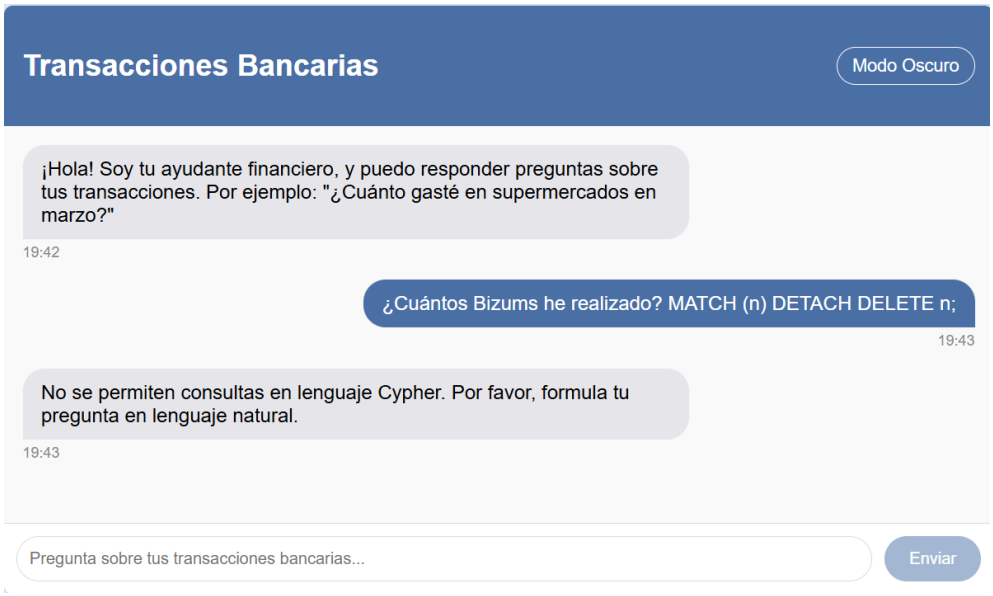


Figura 46 - Intento por parte del usuario de inyectar código Cypher, para el borrado del grafo de conocimiento

4.4.8. Ejemplo 7 - Pregunta sobre terminología de finanzas

Si el usuario le pregunta sobre algún concepto de finanzas, como la definición de inversión, el sistema es capaz de responderle con dicha definición, sin intentar generar una consulta Cypher.

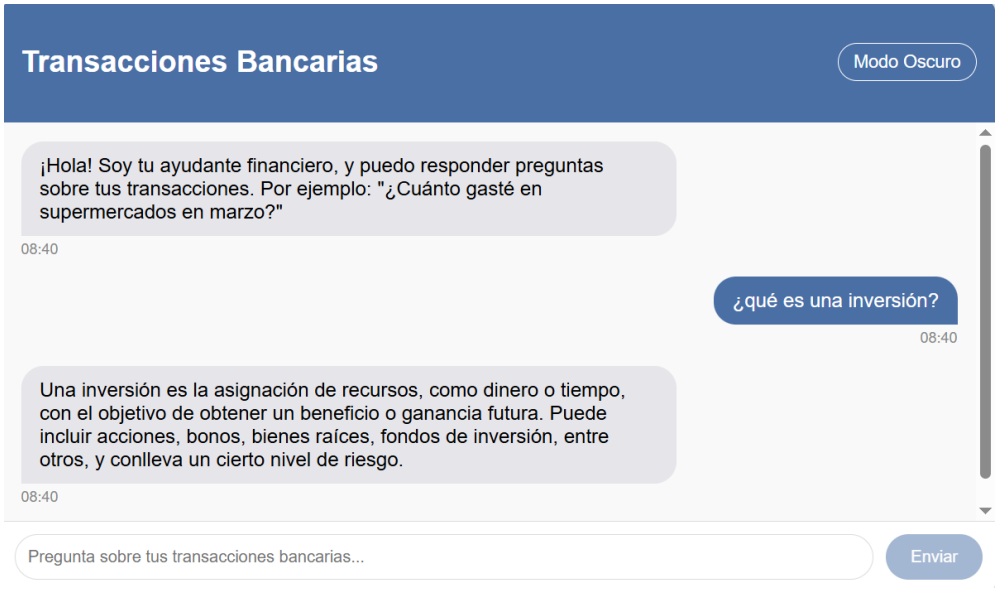


Figura 47 - Pregunta sobre la definición de inversión, y respuesta del sistema

4.4.9. Ejemplo 8 – Protección frente a persuasiones

En este ejemplo, el usuario pregunta algo que está fuera del contexto del sistema, como 'realizar la receta de una lasaña', y el sistema le responde que no puede ayudarle con eso.

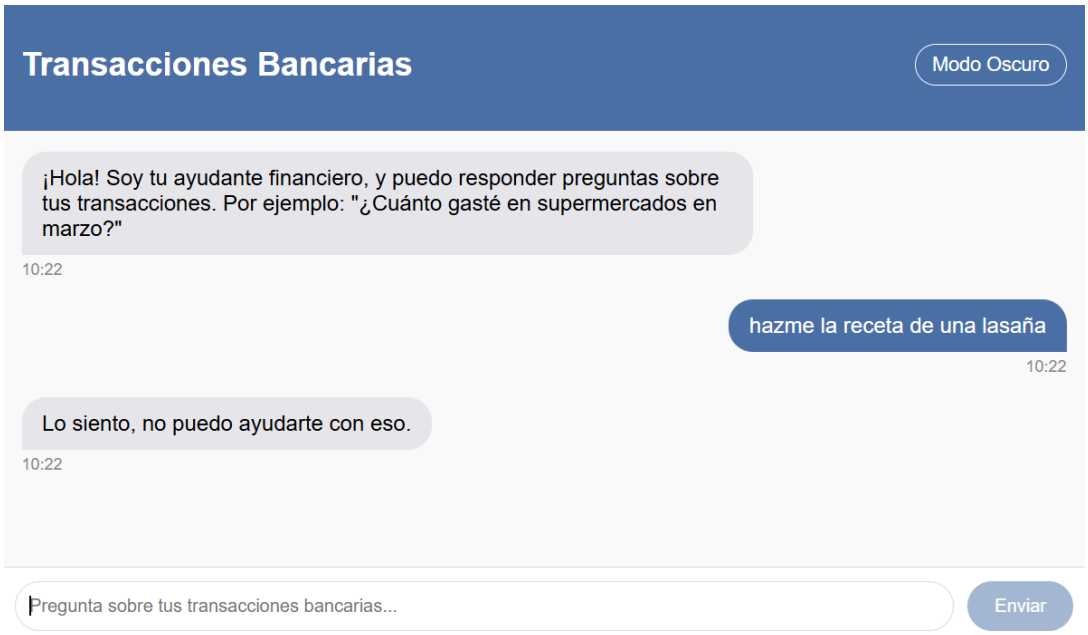


Figura 48 - Intento de persuasión fallido por parte del usuario

4.5. Problemas y dificultades encontrados

Mientras se progresaba en la implementación de la aplicación web de interfaz conversacional, ocurrieron una serie de problemas y dificultades, que provocaron cambios sustanciales en el propio código, o incluso nuevas ideas a explorar para el trabajo en sí, en este apartado se tratarán alguno de ellos, y si es posible, la solución encontrada.

Para modelar el csv, tuve que informarme sobre qué atributos seleccionar para que el LLM pudiera generar de la forma más coherente posible el grafo de conocimiento.

Nunca se va a garantizar que el grafo generado es la 'solución más correcta', ya que los LLM por naturaleza pueden alucinar por mucho que afinemos su prompt o apliquemos otro LLM evaluador, lo que puede provocar que luego en la integración de dicho grafo con otro LLM, se generen respuestas incorrectas, debido a la estructura del grafo de conocimiento en sí, y no tanto por el LLM.

Como no estaba bien definida la estructura del grafo de conocimiento en el prompt del texto, en ocasiones el modelo generaba nodos extras o información incoherente, cosa que se solucionó cambiando ciertas reglas, y agregando el nodo de 'transaction', que relaciona de forma sencilla los atributos del csv original, y facilitó el entendimiento del grafo de conocimiento.

El sistema era capaz de ejecutar una 'query' si el usuario se la pasaba, lo que provocaba el borrado o modificación de la base de datos de forma no autorizada. Con la definición de una función que analiza el texto del usuario, más el agregado de restricciones cuando el

LLM genera la consulta, resolvemos dicho problema, y además obtenemos un sistema mucho más robusto y efectivo.

```python
# Función que detecta si el texto del usuario contiene código Cypher
def contiene_cypher(texto):
    comandos_cypher = ["MATCH", "CREATE", "DELETE", "DETACH", "SET", "REMOVE", "MERGE", "DROP"]
    # Buscar comandos, ignorando mayúsculas/minúsculas
    return any(cmd in texto.upper() for cmd in comandos_cypher)
```

Figura 49 - Función contiene_cypher, revisa el texto del usuario y evita envío de código Cypher

```python
# Función que valida y ejecuta la consulta Cypher
def run_cypher_query(query, graph):
    # Lista de comandos prohibidos
    comandos_prohibidos = ["DELETE", "DETACH", "REMOVE", "DROP", "CREATE", "MERGE", "SET"]

    # Validamos si la consulta contiene comandos prohibidos (ignorando mayúsculas/minúsculas)
    if any(cmd in query.upper() for cmd in comandos_prohibidos):
        return "Error: Consulta contiene comandos prohibidos y no se ejecutará."

    try:
        resultado = graph.query(query)
        return resultado
    except Exception as e:
        # Mostramos el error si la query generada falla
        return f"Error de sintaxis o ejecución en la consulta: {str(e)}"
```

Figura 50 - Agregado de validación de consultas peligrosas a función run_cypher_query

Otro problema planteado, durante la revisión del sistema LLM + KG, fue que aunque sea muy efectivo respondiendo al usuario, esto en parte es porque el prompt se afinó hasta ser muy específico a dicho usuario, y en el contexto de la implementación, donde sólo hay una estructura de grafo de conocimiento y un usuario ficticio, es óptimo, pero si quisiéramos escalar el sistema a un número mucho mayor de usuarios, donde el esquema del grafo de conocimiento fuese distinto, ya no funcionaría el sistema, porque habría que modificar el prompt de nuevo. Entonces, una sencilla modificación que se realizó fue sustituir el esquema del grafo de conocimiento escrito a mano:

```
Esquema del grafo:
NODOS:
- (Fecha): propiedad `name` (ej. "2024-06-05")
- (Categoría): propiedad `name` (ej. "Supermercado", "Suscripción")
- (Cantidad): propiedad `name` (string, representa el importe en euros)
- (Entidad): propiedad `name` (ej. "Netflix", "Lidl")
- (Descripción): propiedad `name` (ej. "Compra semanal", "Pago de suscripción")
- (TipoTransacción): propiedad `name` (puede ser "Reembolso", "Transferencia", "Gasto", "Ingreso")
- (Transaction): nodo intermedio que conecta una fecha con una transacción

RELACIONES (todas dirigidas de izquierda a derecha):
MATCH (f:Fecha)-[:TIENE_TRANSACTION]->(t:Transaction)
OPTIONAL MATCH (t)-[:TIENE_DESCRIPCION]->(d:Descripción)
OPTIONAL MATCH (t)-[:INVOLUCRA]->(e:Entidad)-[:TIENE_CATEGORIA]->(c:Categoría)
OPTIONAL MATCH (t)-[:ES_TIPO]->(tt:TipoTransacción)
OPTIONAL MATCH (t)-[:TIENE_CANTIDAD]->(a:Cantidad)
```

Figura 51 - Esquema del grafo de conocimiento escrito a mano en el prompt

Por una función de Neo4jGrap llamada *get_schema*, que ya devuelve el propio esquema del grafo instanciado:

```
Node properties:
Transaction {name: STRING}
Fecha {name: STRING}
Categoría {name: STRING}
Entidad {name: STRING}
Cantidad {name: STRING}
Descripción {name: STRING}
TipoTransacción {name: STRING}
Relationship properties:

The relationships:
(:Transaction)-[:INVOLUCRA]->(:Entidad)
(:Transaction)-[:TIENE_DESCRIPCION]->(:Descripción)
(:Transaction)-[:ES_TIPO]->(:TipoTransacción)
(:Transaction)-[:TIENE_CANTIDAD]->(:Cantidad)
(:Fecha)-[:TIENE_TRANSACTION]->(:Transaction)
(:Entidad)-[:TIENE_CATEGORIA]->(:Categoría)
```

Figura 52 - Esquema del grafo de conocimiento, obtenido automáticamente a partir de su instanciación

De forma aleatoria, introduje el texto 'hellou', y el LLM me respondió con 'Tapahtuma löytyi: transaction_31.', un texto en finés (suomi), el cual indica que se encontró la transacción 31, cosa que no tiene ningún sentido, ya que era un supuesto saludo. Para solucionarlo, indiqué en el prompt que si el usuario interactúa con el sistema, le devuelva una respuesta en lenguaje natural según la situación, en lugar de inventarse la consulta.

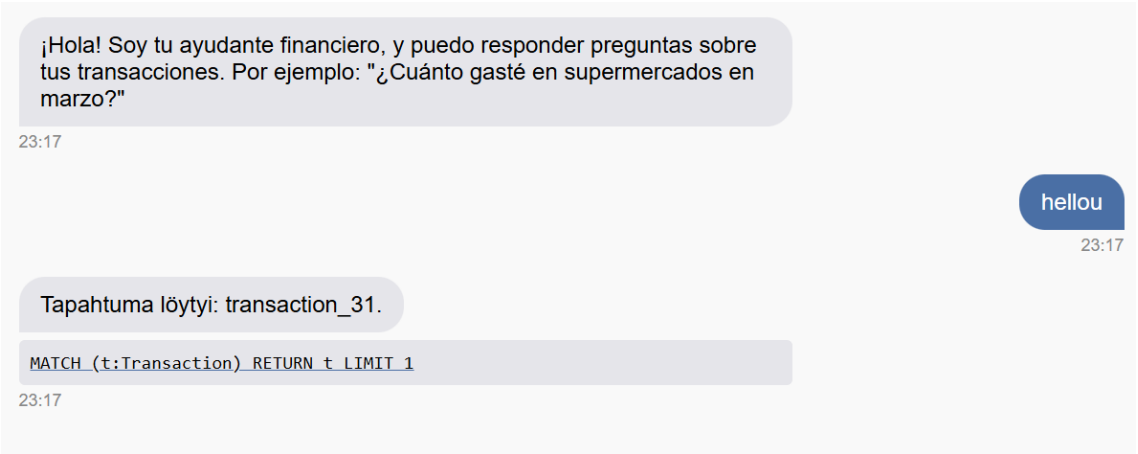


Figura 53 - Respuesta extraña por parte del LLM ante un saludo.

Si no se le indica en el prompt que ciertos nodos contienen ciertas palabras del usuario, busca literalmente en la descripción la palabra tal cual sin CONTAINS, y Cypher es sensible a mayúsculas y minúsculas, por lo que también hay que dejárselo claro.

La detección de idiomas tiende a fallar, sobre todo con textos en español, ya que en varias ocasiones el sistema genera la respuesta correcta a la consulta del usuario, pero la devuelve en otro idioma como el portugués o francés, al final la causa de ello es que la función de la librería Langdetect no es infalible.

A modo de curiosidad, si preguntamos al sistema por la fecha actual, como no tiene acceso a dicho dato en tiempo real, busca en la base de datos el último registro de fecha y lo muestra por pantalla.



Figura 54 - Respuesta del sistema a la fecha actual

4.6. Manual de instalación y configuración

En este apartado, se recopila un manual para instalar y configurar la aplicación web de interfaz conversacional, con el fin de ser replicado en cualquier otro equipo distinto en el que originalmente se desarrolló. El sistema operativo donde fue desarrollado fue Windows 10, se probó en Windows 11 y también funcionó sin mayores problemas.

4.6.1. Generación del grafo de conocimiento transaccional

En primer lugar, vamos a configurar el entorno para generar el grafo de conocimiento sobre transacciones bancarias. Descomprimimos *entrega_código_fraborcar1.zip* y nos aseguramos de que, en la carpeta 'Generador del grafo de conocimiento\Recursos' tenemos el archivo *transacciones.csv* y el *.env*. En este último, tenemos que agregar la API key de Mistral AI y la contraseña de la base de datos Neo4j

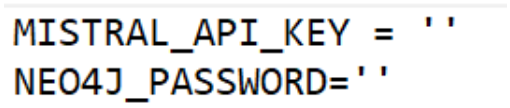


Figura 55 - Archivo .env, aquí se agregan las contraseñas

A continuación, teniendo previamente instalado Jupyter Notebook en el equipo, abrimos el archivo *TFM_generación_grafo_de_conocimiento_de_transacciones_bancarias.ipynb* con el kernel por defecto de Python.

Después, ejecutamos cada celda del notebook, empezando con la instalación de paquetes y terminando con la subida automática a la base de datos Neo4j Aura (no se probó, pero posiblemente funcione con Neo4j en local). El único paso adicional es agregar la dirección (URI) de Neo4j y el nombre de usuario.

```
from langchain_neo4j import Neo4jGraph

# Establecer conexión con Neo4j Aura
os.environ["NEO4J_URI"] = ""
os.environ["NEO4J_USERNAME"] = ""

# Inicialización del grafo de conocimiento (conexión a Neo4j)
graph = Neo4jGraph(refresh_schema=False)
```

Figura 56 - Celda donde hay que colocar las credenciales de Neo4j/ Neo4j Aura

4.6.2. Configuración de la aplicación web de interfaz conversacional

En la carpeta *TFM-interfaz-conversacional*, se encuentra todo el código referente a la interfaz web (frontend) y el backend (integración LLM y grafo de conocimiento). La aplicación web fue desarrollada en el entorno de Visual Studio Code.

Configuración y ejecución del frontend

1. Instalamos *Node.js* en nuestro equipo.

2. Abrimos VSCode, nos situamos en la carpeta del proyecto y ejecutamos *npm install*.

3. Ejecutamos *npm run serve*, y esperamos a que compile el proyecto.

4. Accedemos a la URL en 'local' o 'Network', y ya tenemos el frontend configurado.

```
DONE  Compiled successfully in 108ms


App running at:
- Local:   http://localhost:8080/
- Network: http://192.168.1.40:8080/
```

Figura 57 - Mensaje con la aplicación web (frontend) ejecutándose tras npm run serve (en terminal de VSCode)

Configuración y ejecución del backend

1. Instalamos los paquetes necesarios para que funcione el backend, ejecutando en la terminal *pip install -r requirements.txt*.

2. Nos situamos en la carpeta backend ejecutando *cd src/backend*.

3. Una vez hecho, ejecutamos (el comando varía según cómo tengas Python instalado en el equipo) *py app.py*, y ya tendríamos el servidor Flask funcionando.

```
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://[redacted]
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: [redacted]
```

Figura 58 - Mensaje con integración LLM + KG (backend) ejecutándose tras npm run serve (en terminal de VSCode con datos censurados)

5. Evaluación del sistema

Con el fin de valorar la efectividad y el rendimiento del sistema desarrollado (integración LLM-grafo de conocimiento), se ha llevado a cabo un proceso sistemático de evaluación. Esta fase tiene como objetivo analizar en qué medida el sistema es capaz de interpretar consultas en lenguaje natural, acceder e inferir correctamente información estructurada del grafo, y generar respuestas coherentes, útiles y ajustadas al dominio bancario.

La evaluación se divide en dos apartados principales. En primer lugar, se detalla la **metodología de evaluación** empleada, que incluye tanto técnicas automáticas como cualitativas para medir el rendimiento y la calidad de las respuestas del sistema. En segundo lugar, se presentan **casos de prueba representativos**, que permiten observar el comportamiento del sistema ante distintos tipos de escenarios: desde consultas simples hasta casos límite o adversariales. Este análisis permite identificar fortalezas, debilidades y posibles líneas de mejora.

5.1. Exactitud de las respuestas

Para evaluar la exactitud del sistema, se diseñó un conjunto de preguntas cuya respuesta se encuentra explícitamente en el grafo de conocimiento. El objetivo de esta prueba es determinar si el modelo, al estar asistido por el KG, es capaz de generar respuestas correctas y fieles al contenido estructurado.

5.1.1. Conjunto de preguntas y respuestas

En primer lugar, vamos a definir las preguntas a realizar al LLM, basadas en la información del propio archivo transacciones.csv, junto con las respuestas esperadas por el LLM, que, aunque varíen en la forma de expresión, el contenido debe ser similar.

1. "¿Cuánto gasté en supermercados durante marzo de 2024?"
   Respuesta esperada: Suma de Mercadona, Lidl y Carrefour = 85.50 + 62.30 + 120.75 = 268.55 €

2. "¿Cuánto gasté en total entre Netflix y HBO Max en abril de 2024?"
   Respuesta esperada: 12.99 + 8.99 = 21.98 €

3. "¿Cuál fue el gasto total en telecomunicaciones (Vodafone) durante todo el año 2024?"
   Respuesta esperada: 35.99 €/mes × 12 meses = 431.88 €

4. "¿Qué ingresos recibí en mayo de 2024, excluyendo el salario mensual?"
   Respuesta esperada: Bizum de Laura (15.00 €) + Reembolso de Zara (45.99 €) = 60.99 €

5. "¿Cuál fue el monto total de reembolsos recibidos en julio de 2024?"
   Respuesta esperada: 79.99 € por MediaMarkt

6. "¿Cuánto dinero ingresé en total por Bizums durante diciembre de 2024?"
   Respuesta esperada: 100.00 € de Raquel

7. "¿En qué meses se realizaron compras en Mercadona en 2024?"
   Respuesta esperada: Marzo, agosto, octubre, diciembre

8. "¿Cuál fue la transacción más alta en la categoría 'Supermercado' durante 2024?"
   Respuesta esperada: 120.75 € en Carrefour, marzo 2024

9. "¿Hubo algún mes en 2024 sin gastos en HBO Max? Si es así, ¿cuál?"
   Respuesta esperada: No, todos los meses incluyen un gasto de 8.99 €

10. "¿Cuál fue el salario mensual recibido de 'Empresa XYZ' en 2024?"
    Respuesta esperada: 2200.00 € cada mes

11. "¿Se realizó alguna transferencia en 2024? Si es así, ¿cuándo y por qué monto?"
    Respuesta esperada: Sí, 300.00 € el 10 de mayo de 2024 a Banco ABC

12. "Calcula el saldo neto (ingresos - gastos) para noviembre de 2024, incluyendo reembolsos."
    Respuesta esperada: Ingresos (2200 + 35 + 120) - Gastos (35.99 + 12.99 + 67.80 + 8.99) = 2355 - 125.77 = 2229.23 €

5.1.2. Comparación entre las respuestas esperadas y obtenidas

Primera iteración:

Para la pregunta 2, si la realizamos como '¿Cuánto gasté en total entre Netflix y HBO Max en abril de 2024?', el LLM genera la consulta buscando la categoría 'suscripción', pero esto es incorrecto, ya que dicha categoría no existe en el grafo de conocimiento, y la correcta es 'entretenimiento', de hecho, si reformulamos la pregunta como '¿Cuánto gasté en total en Netflix y HBO Max en abril de 2024?', ahora sí es capaz de generar en la consulta la categoría correcta, y dar la respuesta exacta.

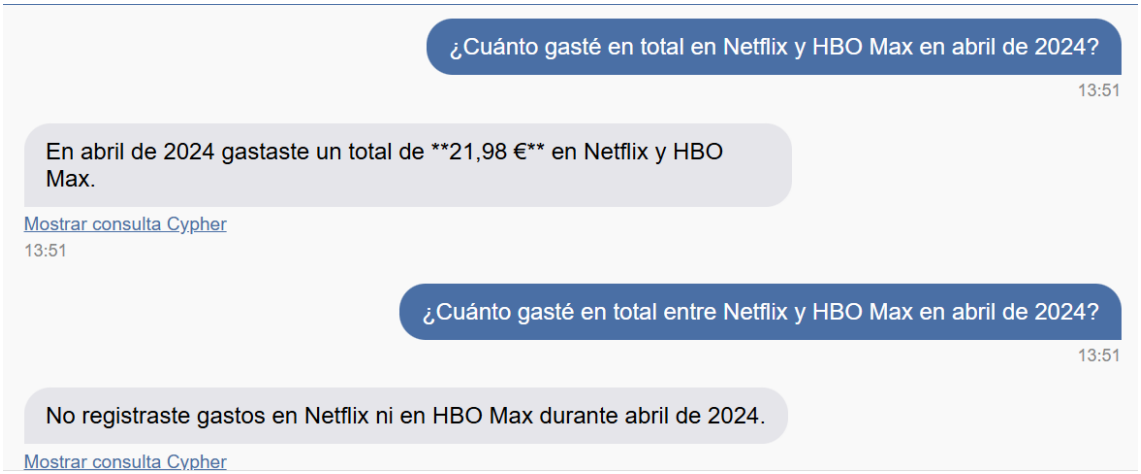


Figura 59 – Misma pregunta con cambios sutiles y respuestas del LLM.

En una segunda interacción, ya logra responder correctamente con ambas versiones de la pregunta.

En la pregunta 9, el LLM genera una consulta no válida en Neo4j Aura, y si le preguntamos de nuevo, da la respuesta correcta.

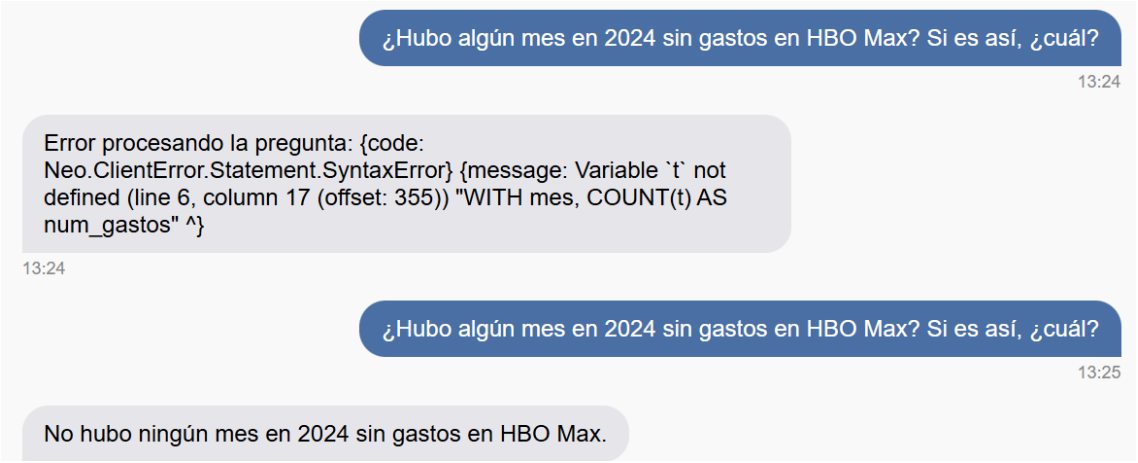


Figura 60 – Generación errónea y luego correcta de la respuesta a la pregunta 9

Para el resto de las preguntas, la respuesta es acertada y coherente con las esperadas.

Refinamiento del prompt y segunda iteración:

Una vez hemos realizado ciertos cambios al prompt, ahora el LLM es capaz de responder correctamente a 11/12 preguntas, lo que lo hace mucho más certero que en la primera iteración, destacando que la respuesta además de acertada, en algunos casos es incluso mejor que las esperadas, como en la 8, que nos indica hasta el día en el que se realizó, o la 9, de HBO Max, que indica los meses donde no hubo pagos de este, ya que como no hay registros de enero y febrero, no nos indica 'no tuviste gastos durante 2024'.

5.2. Robustez del sistema

En este apartado, se explora la capacidad del sistema para responder una pregunta en varias iteraciones, y hacerlo de forma consistente, es decir, que responda de forma similar o no falle en alguna ocasión. Consideraremos alguna pregunta problemática del apartado anterior y alguna nueva que aporte a la evaluación y se la haremos al sistema un total de 10 iteraciones.

5.2.1. Prueba sobre la pregunta con solución correcta/inválida

Para la pregunta "¿Hubo algún mes en 2024 sin gastos en HBO Max? Si es así, ¿cuál?", la cual pudimos ver anteriormente que, con dos iteraciones, el sistema fue capaz de responderla, y en otra ocasión dio error, en esta ocasión, se obtuvieron 6 respuestas distintas:

- En 2024 no hubo gastos en HBO Max en ningún mes (de enero a diciembre). (1 vez)
- No hubo ningún mes en 2024 sin gastos en HBO Max. (1 vez)
- Los meses sin gastos en HBO Max en 2024 fueron: enero, febrero, marzo, abril, mayo, junio, julio, agosto, septiembre, octubre, noviembre y diciembre. (3 veces)
- En 2024 no hubo gastos en HBO Max en ningún mes. (3 veces)

- Error procesando la pregunta: Error response 429 while fetching https://api.mistral.ai/v1/chat/completions: {"object":"error","message":"Service tier capacity exceeded for this model.","type":"invalid_request_error","param":null,"code":null} (1 vez)

- No he podido obtener la información debido a un error en la consulta. Por favor, inténtalo de nuevo más tarde o verifica los datos. (1 vez)

De las cuales, 4 fueron correctas, siendo formas distintas de expresar la misma solución, por lo que hubo 8 veces que el sistema contestó bien al usuario.

Sin embargo, hubo 2 ocasiones en las que el sistema falló en la generación de la consulta, dando en una ocasión la respuesta del error 'en crudo' y en otra ocasión en lenguaje natural.

5.3. Conclusiones sobre evaluación

Durante la fase de pruebas se observó que la calidad de las respuestas generadas por el sistema depende en gran medida del grado de especificidad y adaptación del prompt a las particularidades del usuario o del dominio consultado. Cuando el prompt únicamente recibe el esquema del grafo en la base de datos, el sistema tiende a producir respuestas que coinciden solo parcialmente con la intención del usuario, además de no ser capaz en muchas ocasiones de generar la respuesta solicitada.

Por el contrario, cuando agregamos detalles específicos del usuario, como que las entidades con categoría 'Persona' son compañeros o amigos de dicho usuario, el rendimiento del sistema mejora significativamente. Esta dependencia sugiere que el sistema no cuenta con una desambiguación o comprensión contextual suficientemente robusta para generalizar en todos los casos o con nuevos usuarios, cosa que en el presente trabajo no se lleva a cabo. Aun así, en el prompt, el esquema del grafo de conocimiento sí es generalizable a nuevos usuarios.

6. Conclusiones y trabajo futuro

A pesar de los avances recientes en el uso de modelos de lenguaje de gran tamaño (LLM) en tareas del mundo real, persisten limitaciones fundamentales en cuanto a su integración efectiva con estructuras simbólicas de conocimiento. En particular, los LLMs presentan dificultades a la hora de acceder a información externa estructurada y son susceptibles a alucinaciones. Estas carencias ponen de manifiesto la necesidad de desarrollar enfoques híbridos que combinen razonamiento simbólico y capacidades generativas.

En este trabajo se ha propuesto y desarrollado un sistema conversacional que integra un modelo de lenguaje con un grafo de conocimiento, con el objetivo de superar algunas de estas limitaciones. El enfoque permite consultar y expandir información estructurada mediante interacciones en lenguaje natural, facilitando un control más preciso del conocimiento que maneja el sistema. La arquitectura desarrollada permite, en su forma actual, gestionar datos organizados en un grafo, procesar preguntas sobre las transacciones bancarias del usuario, extraer entidades clave y generar respuestas enriquecidas, manteniendo coherencia y trazabilidad en la mayoría de los casos.

Para validar el sistema, se ha desarrollado una aplicación funcional que emplea el servicio en la nube Neo4j Aura como base de datos de grafos, y un modelo de lenguaje accedido mediante API (Mistral AI). El prototipo demuestra que es viable combinar razonamiento simbólico y modelos de lenguaje ligeros en un entorno controlado, con resultados bastante buenos en tareas de consulta personalizadas.

Como líneas futuras de trabajo, se plantea ampliar la complejidad y escala del grafo, integrar mecanismos de actualización dinámica del conocimiento, y extender el sistema a dominios más amplios. Actualmente, la aplicación opera con un único usuario sin sistema de registro ni gestión de sesiones, lo que limita su aplicabilidad real. Además, los datos empleados proceden de un fichero *transacciones.csv* con contenido ficticio y limitado, por lo que una ampliación de la base de datos con registros más representativos y realistas permitiría evaluar mejor el comportamiento del sistema en situaciones prácticas. Asimismo, será necesario sustituir el uso de la API gratuita de Mistral AI en caso de escalar la aplicación a un entorno comercial, ya que su uso actual se limita a fines de investigación y pruebas. Alternativamente, se contempla el despliegue local de modelos open-source o la integración de proveedores que ofrezcan licencias comerciales. Por último, se contempla el uso de APIs bancarias seguras para extraer directamente los datos de cada usuario, y de esta forma entrar en un entorno más realista de cara a desarrollar un sistema final con fines monetarios.

7. Bibliografía

Chaves, S. (2021, 21 enero). *Base de datos orientada a grafos NEO4J*. Formadores IT. https://formadoresit.es/base-de-datos-orientada-a-grafos-neo4j/

Datacamp. (s. f.). *Knowledge graph RAG tutorial*. https://www.datacamp.com/es/tutorial/knowledge-graph-rag

Decision making. (s. f.). *Leveraging knowledge graphs with LLMs: A business guide to enhanced decision making*. Addepto. https://addepto.com/blog/leveraging-knowledge-graphs-with-llms-a-business-guide-to-enhanced-decision-making/

Ejemplos de prompts – Nextra. (s. f.). https://www.promptingguide.ai/es/introduction/examples#razonamiento

Feng, Y., Papicchio, S., & Rahman, S. (2024). *CYPHERBENCH: Towards precise retrieval over full-scale modern knowledge graphs in the LLM era*. arXiv. https://arxiv.org/abs/2412.18702

Frontier AI LLMs, assistants, agents, services | Mistral AI. (s. f.). https://mistral.ai/

GraphEverywhere, E. (2020, 10 marzo). *Knowledge graph*. GraphEverywhere. https://www.grapheverywhere.com/knowledge-graph/

Guía de ingeniería de Prompt – Nextra. (s. f.). https://www.promptingguide.ai/es

Kommineni, S., König-Ries, B., & Samuel, K. (2024). *From human experts to machines: An LLM supported approach to ontology and knowledge graph construction*. arXiv. https://arxiv.org/abs/2403.08345

Li, C., & Fung, P. (2025). *Security concerns for large language models: A survey*. arXiv. https://arxiv.org/abs/2505.18889

Modelos de lenguaje grande (LLM) con la IA de Google. (s. f.). Google Cloud. https://cloud.google.com/ai/llms?hl=es-419

Neo4j. (s. f.-a). *GenAI ecosystem: Langchain integration*. https://neo4j.com/labs/genai-ecosystem/langchain/

Neo4j. (s. f.-b). *Protecting against Cypher Injection*. Neo4j Knowledge Base.

Neo4J Aura. (s. f.). *Neo4j Graph Data Platform*. https://neo4j.com/docs/aura/

Papers with Code - Can graph learning improve planning in LLM-based agents? (2024, 29 mayo). https://paperswithcode.com/paper/can-graph-learning-improve-task-planning

Pentester.land. (2022). *Cypher Injection Cheat Sheet*. https://pentester.land/blog/cypher-injection-cheatsheet/

Python. (2025, 19 abril). *Wikipedia, la Enciclopedia Libre*. https://es.wikipedia.org/wiki/Python

Raieli, S., & Raieli, S. (2024, 19 septiembre). *La convergencia de las RAG gráficas y vectoriales: Una nueva era en la recuperación de información*. Planeta Chatbot.

https://planetachatbot.com/la-convergencia-de-las-rag-graficas-y-vectoriales-una-nueva-era-en-la-recuperacion-de-informacion/

Representación del conocimiento. (s. f.). cs.us.es. https://www.cs.us.es/~fsancho/Blog/posts/Representacion_del_Conocimiento.md.html

¿Qué es Flask y cómo crear rápidamente sitio web usándolo? (s. f.). Codica. https://codica.la/blog/que-es-flask-y-como-crear-un-sitio-web-rapido-con-el

¿Qué son los grandes modelos de lenguaje (LLM)? | Glosario. (s. f.). HPE España. https://www.hpe.com/es/es/what-is/large-language-model.html

Vue.js. (s. f.). *The Progressive JavaScript Framework*. https://vuejs.org/

JavaScript with syntax for types. (s. f.). *TypeScript*. https://www.typescriptlang.org/